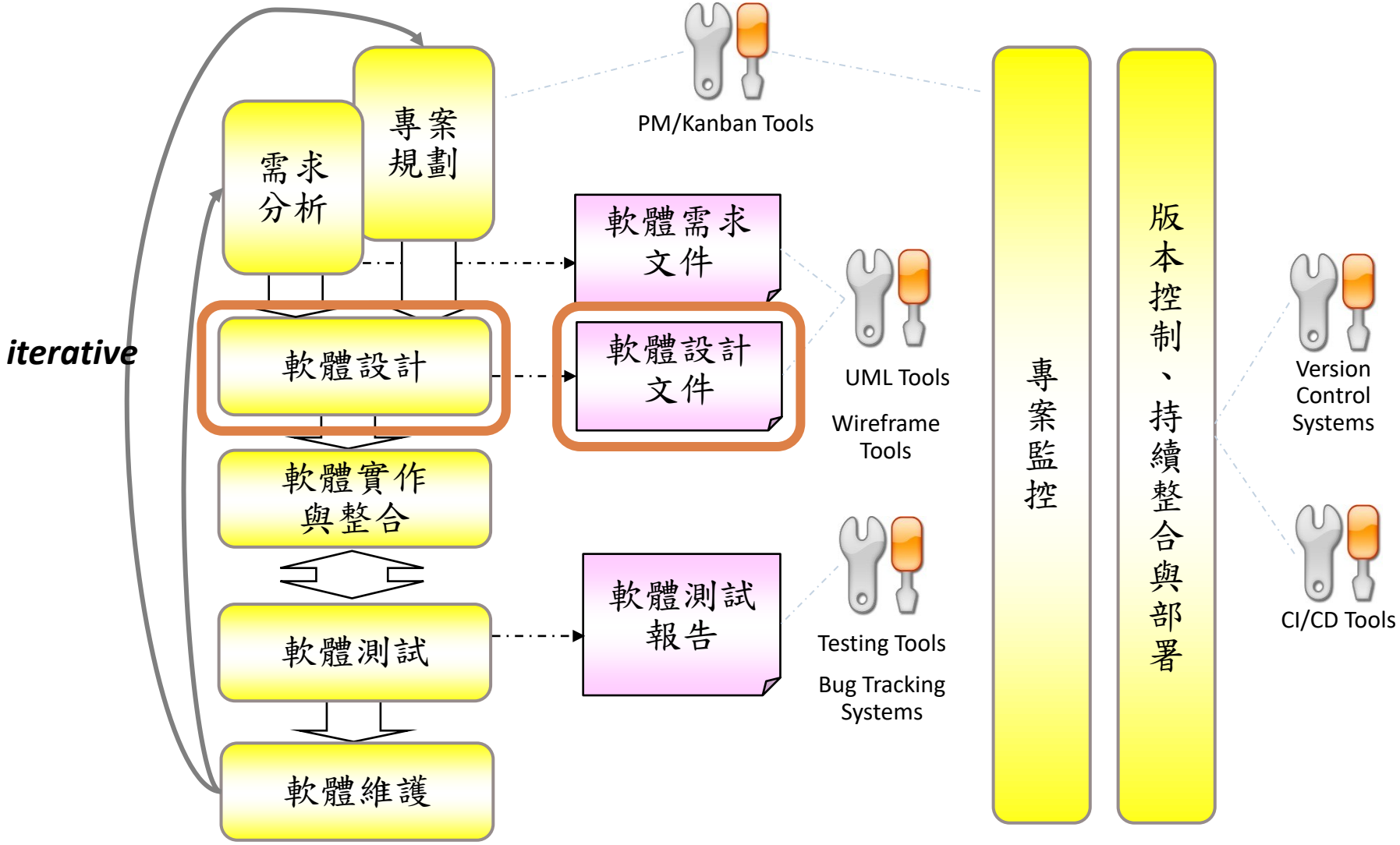
軟體工程課程:
軟體設計-系統設計

海大資工 馬尚彬

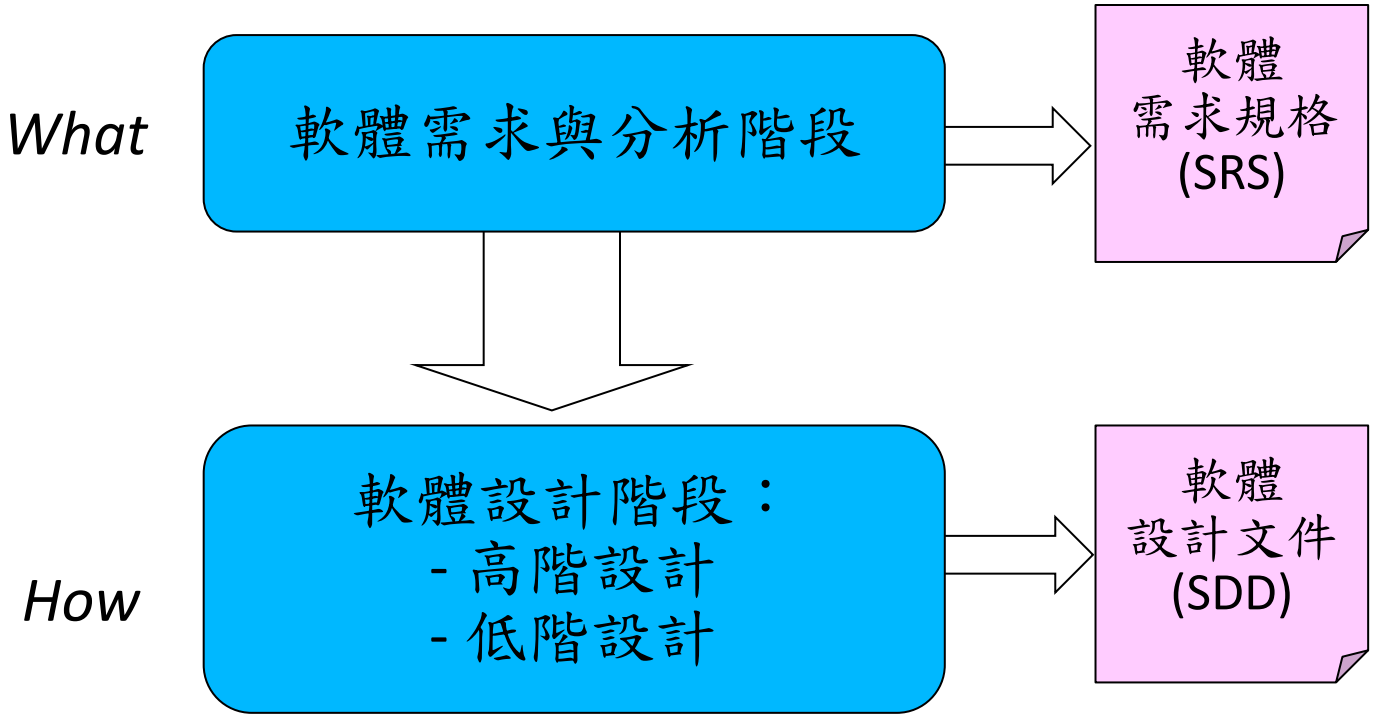
大綱

- 軟體設計基本概念
- 架構設計模式
- 系統設計流程

軟體開發步驟、文件與工具



軟體設計概念



不好的設計：現實世界類別[1]



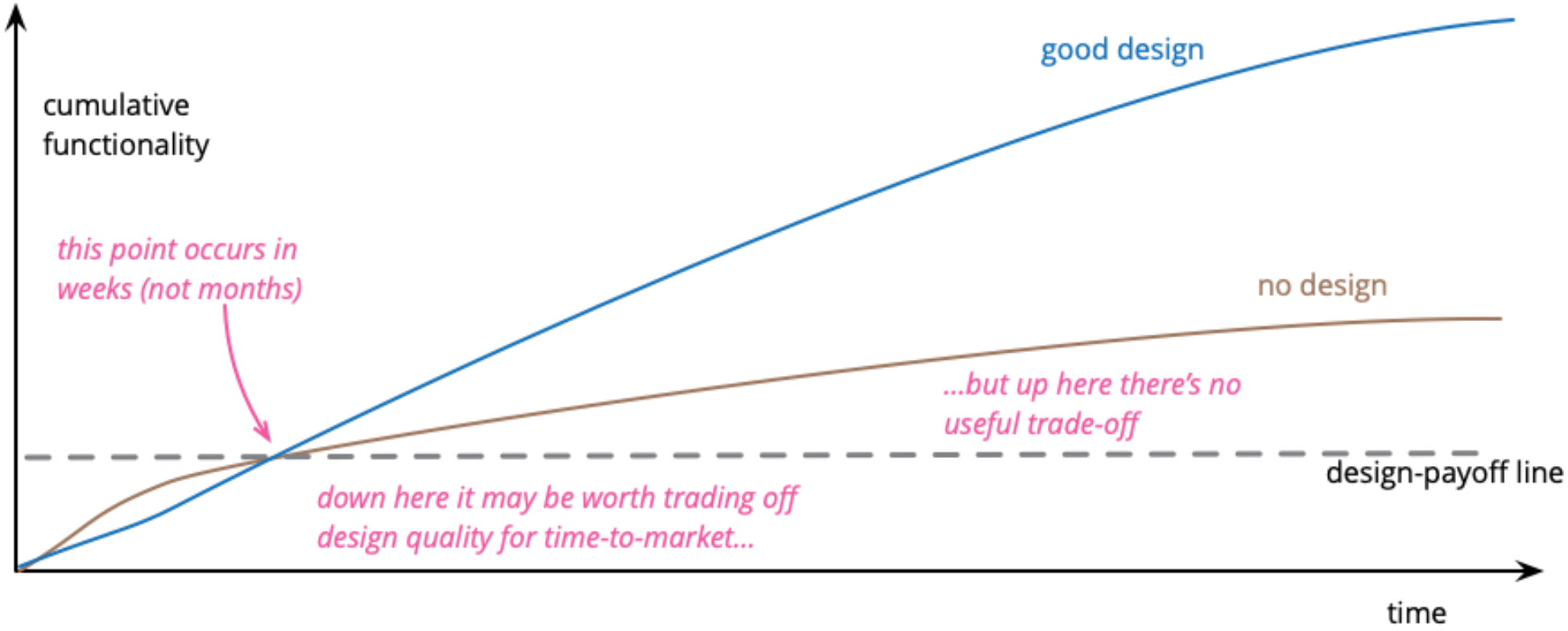
不好的設計：現實世界類別2



不好的設計：現實世界類別3



Good Design V.S. Bad Design



https://martinfowler.com/bliki/DesignStaminaHypothesis.html

抽象化1

- 抽象化(Abstraction)：由許多物件(Entities)中抽離出重要的特性(Features)來，而這些特性，足以讓被抽象化的物件，與別的物件分別開來。
  - 對一般汽車使用者而言，所感興趣的部分是車子的外觀、顏色、車子內裝、安全性、具備哪些功能等，而對於底盤的形式、油管的構造、引擎的設計等較無興趣。
  - 對於汽車研發製造者而言，其關注的焦點在於底盤、油箱、引擎的設計與結合上面，對於外觀、顏色等資訊則可能暫時不列在其關心之列。
- 抽象化階層(Levels of Abstraction)：對於不同的階層會提供不同的抽象化程度。

抽象化2

- 軟體系統發展的抽象化階層：
  - 軟體開發的初期（需求分析階段）：了解整個軟體系統會提供什麼樣的功能給使用者，而對於系統的內部運作則不是這階段的重點。
  - 設計的階段：著重了解軟體內部的設計概念，包含會使用什麼樣的軟體架構，以什麼資料結構以及演算法來實作軟體系統的功能等。
  - 軟體實作(Coding)階段：處理所使用程式語言的細節。

抽象化3

- 軟體設計的抽象化方式
  - 程序抽象化(Procedure Abstraction)
    - 一個特定且具備限制功能的指令程序。
  - 資料抽象化(Data Abstraction)
    - 描述一個資料物件的集合。
    - 抽象資料型態(Abstract Data Type)：C語言中的struct或是C++/Java語言中的class。

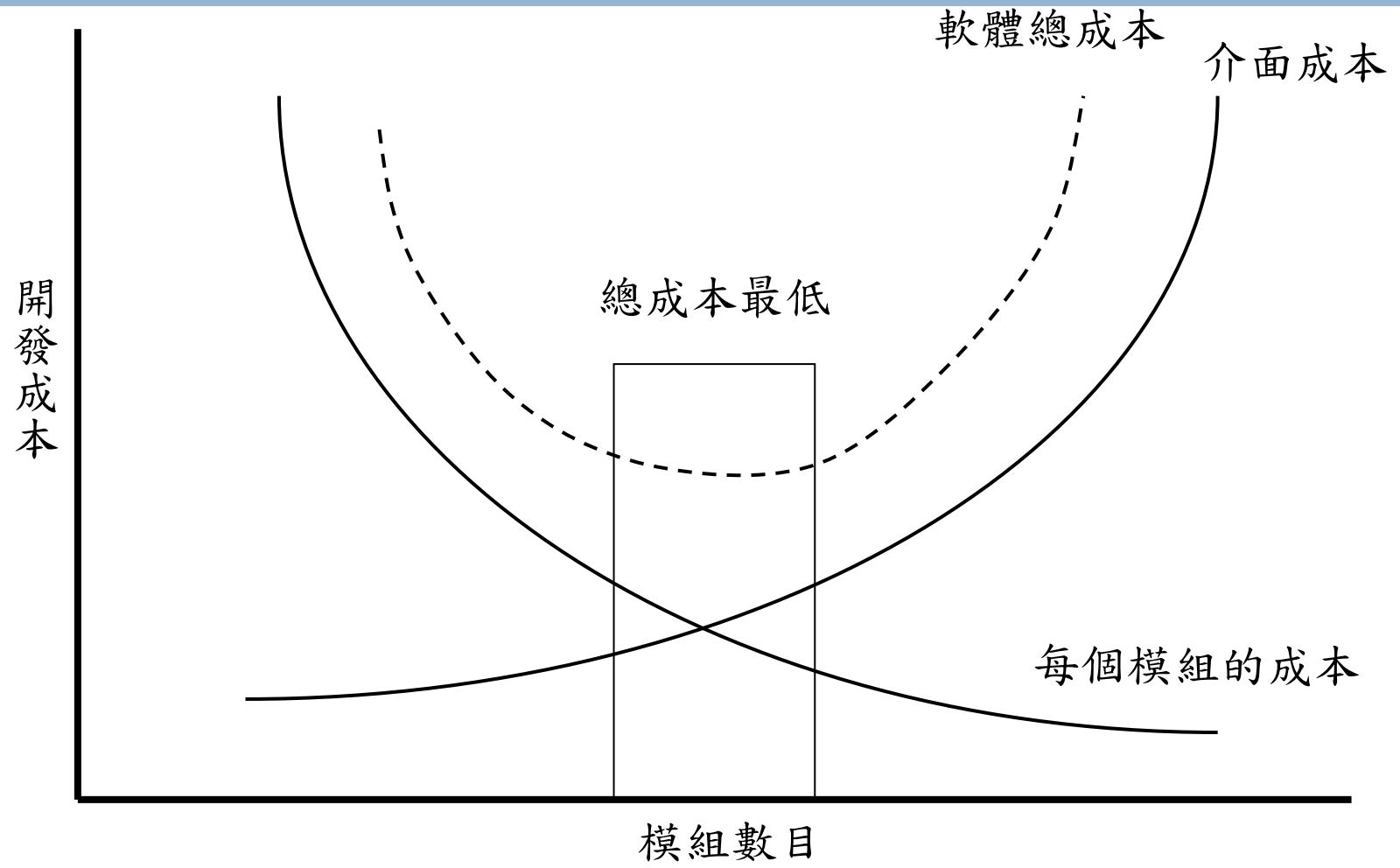
模組化1

- 模組化(Modularity)：將軟體系統依其系統功能性或資料的相依性，分成數個容易了解與處理，而且可以互相溝通的單元
  - 軟體單元：通常稱為模組(Module)或元件(Component)。
    - Standalone piece of code that provides specific and tightly coupled functionality.
    - To define and enforce logical boundaries in your code.
  - 透過軟體系統的模組化可以降低系統的複雜度，提高系統的可維護性，因此，可增加系統的穩定性以及可再利用性(Reusability)。

模組化2

- 模組的大小以及數量：
  - 系統模組愈小，設計就愈容易，但會增加系統模組的數量，增加溝通的成本。
  - 提高模組大小，可減少系統模組的數量與溝通成本，但會增加模組的設計困難度，也降低模組的再利用性。
  - 就系統設計而言，決定模組的大小以及數量是種權衡分析的結果。

模組化3

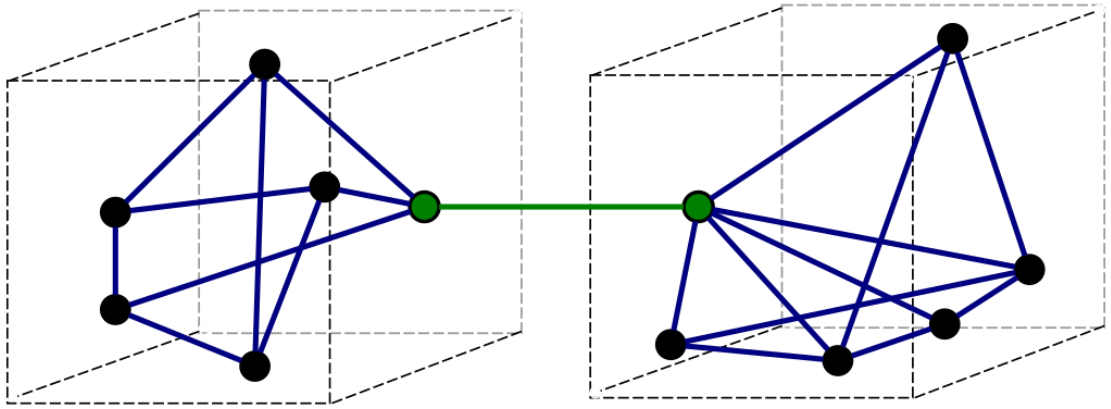


模組與成本關係圖

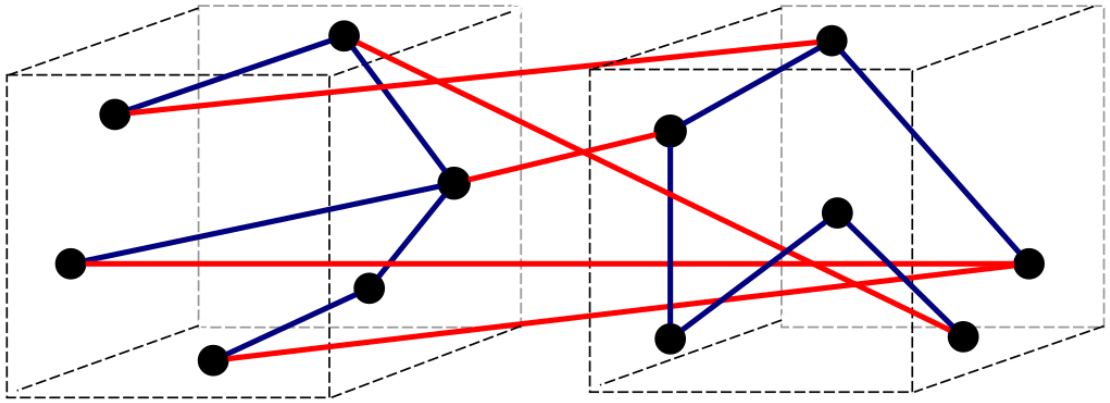
凝聚力與耦合力1

- 凝聚力(Cohesion)：模組內部功能或資料的相依程度。
  - 凝聚力愈高，表示模組內部的功能或資料的相依程度愈高，通常較易維護並且適合再利用。
  - 凝聚力愈低，表示模組內部的功能或資料的相依程度愈低，將增加維護的成本，同時不利模組的再利用。
- 耦合力(Coupling)：模組之間的獨立程度。
  - 耦合力愈低，表示模組間的獨立程度愈高，較易維護並且適合再利用。
  - 耦合力愈高，表示模組間的獨立程度愈低，易增加系統溝通的錯誤，提高維護的成本。

凝聚力與耦合力2



a) Good (loose coupling, high cohesion)



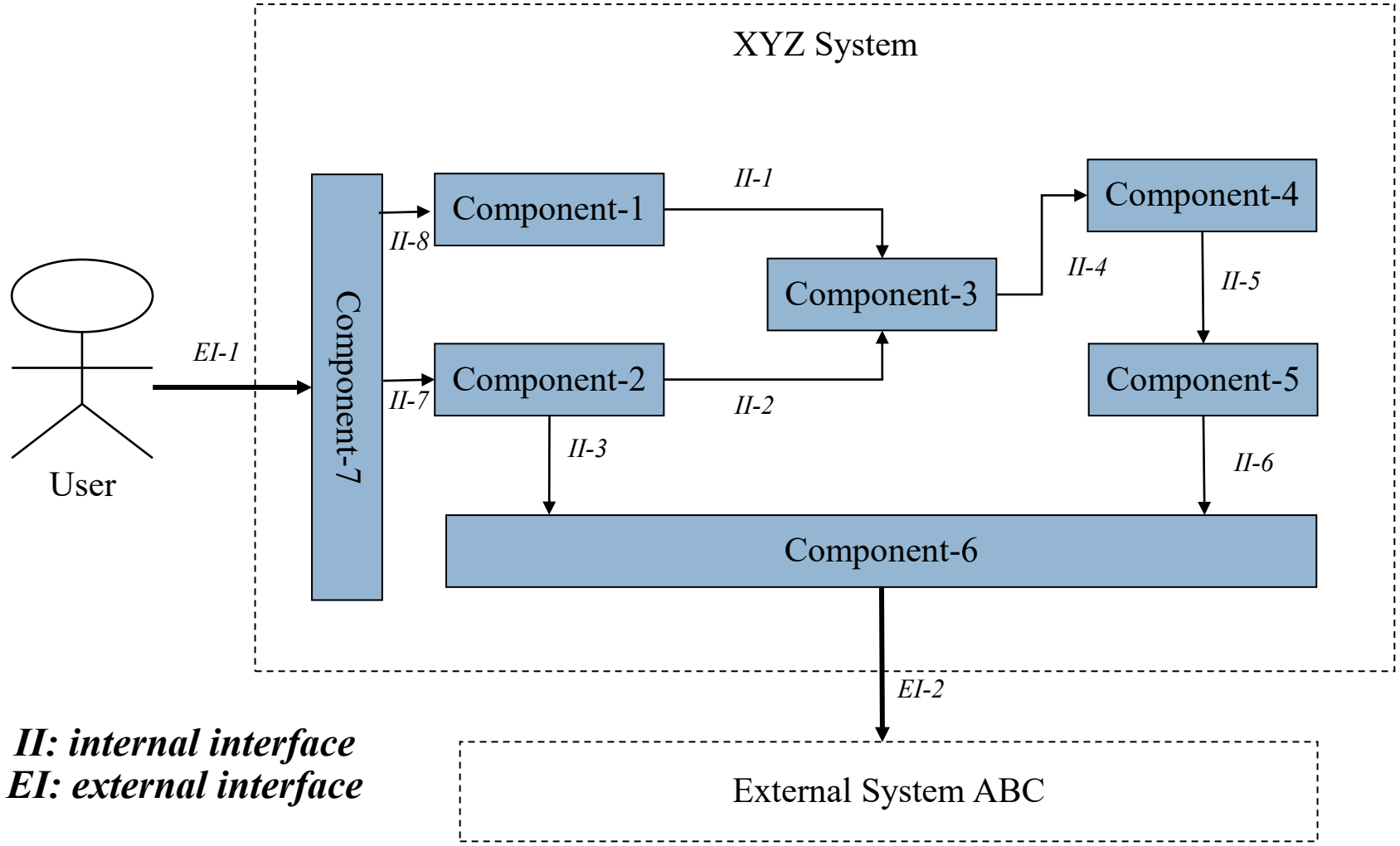
b) Bad (high coupling, low cohesion)

希望達到
"High Cohesion, Low/Loose Coupling"
(高凝聚力、低耦合力)

https://en.wikipedia.org/wiki/Coupling_%28computer_programming%29

架構(Architecture)

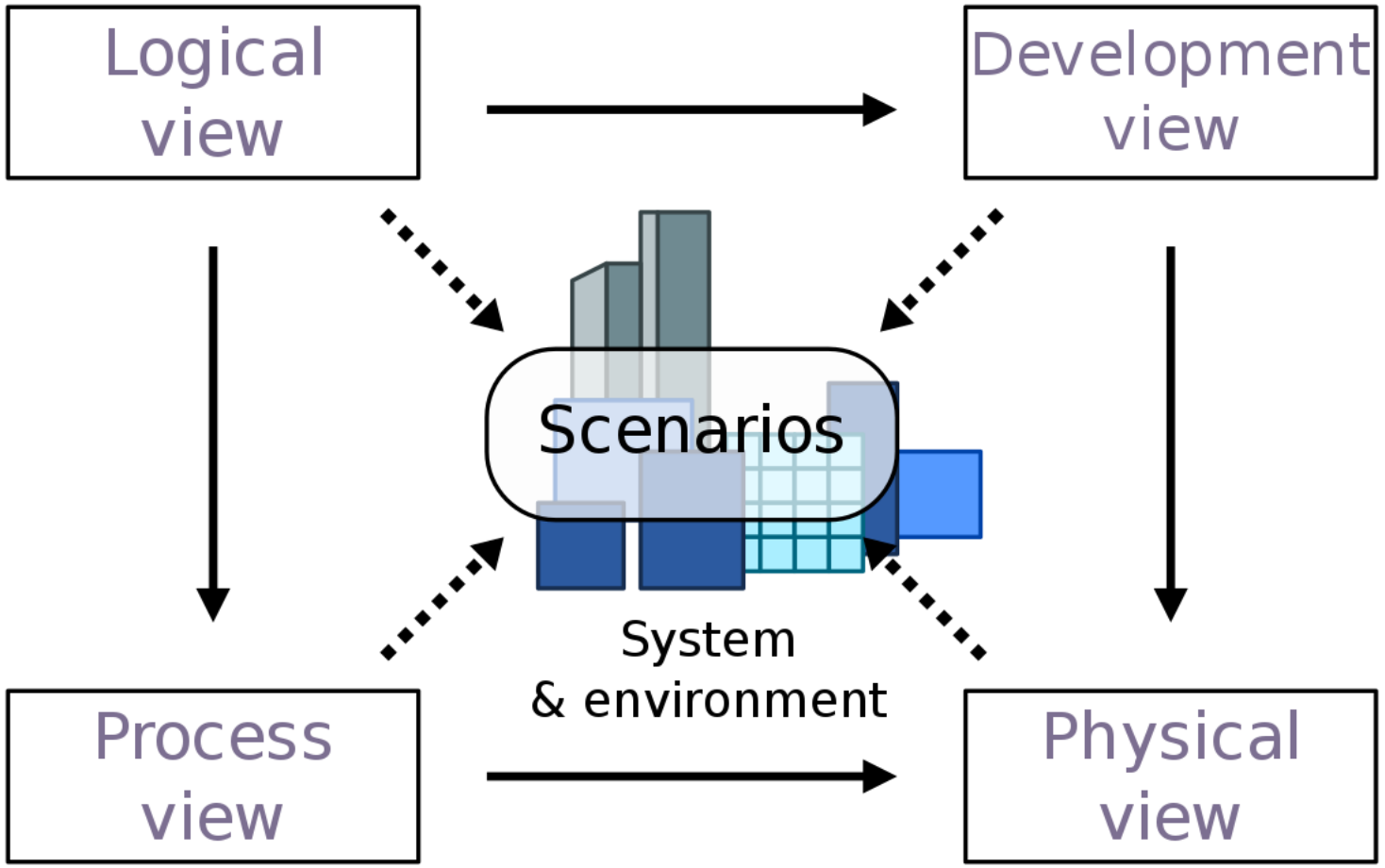
- 架構
  - 從每個層級往下切割一層，例如從system往下切出subsystem，或system往下直接切component/module



架構設計

- 軟體架構設計是架構整體系統組織的過程，包含以下幾個活動：
  - (1)將系統分割成數個子系統/元件/模組
  - (2)決定這些子系統/元件/模組如何互動
  - (3)決定這些子系統/元件/模組的介面
- 軟體架構的構面包含靜態面與動態面：
  - 架構的靜態面：架構設計展現系統切割成不同的子系統/元件/模組，以及之間介面連結。
  - 架構的動態面：架構設計可以描述子系統/元件/模組之間的動態互動行為，資料如何在各個子系統/元件/模組間傳遞與處理，以及其執行的動態行為等。

架構設計：4 + 1 View Model[1]



架構設計：4 + 1 View Model[2]

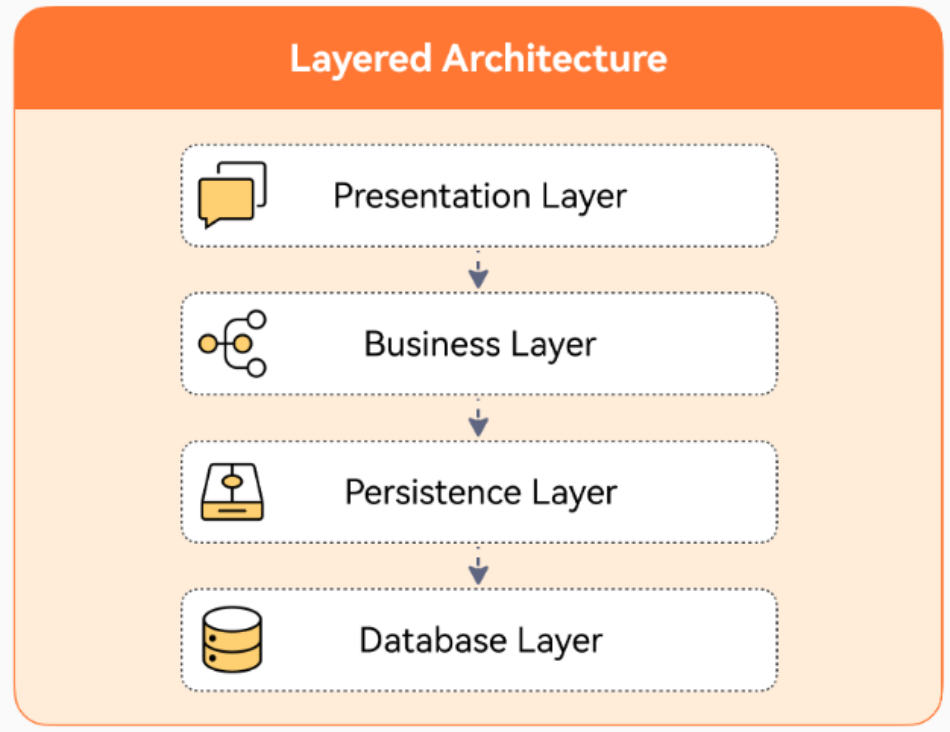
- Logical View
  - Shows the **key abstractions** in the system as objects or object classes. (概念區塊切割)
- Process view
  - Shows how, at **run-time**, the system is **composed of interacting processes**. (考慮系統運作流程)
- Development view
  - Shows how the software is **decomposed for development**. (考慮開發面)
- Physical view
  - Shows the system hardware and **how software components are distributed** across the processors in the system. (考慮部署方式)
- Use cases or Scenarios
  - +1 (考慮使用方式)

架構模式(Architectural Pattern)

- 模式(或樣式)是呈現、共享和重用知識的方法(a means of representing, sharing and reusing knowledge)。
- 架構模式是對良好架構設計實踐方法的描述。
  - 這些架構設計風格已在不同情境下進行了完善的嘗試和測試。

架構模式- 階層架構模式[1]

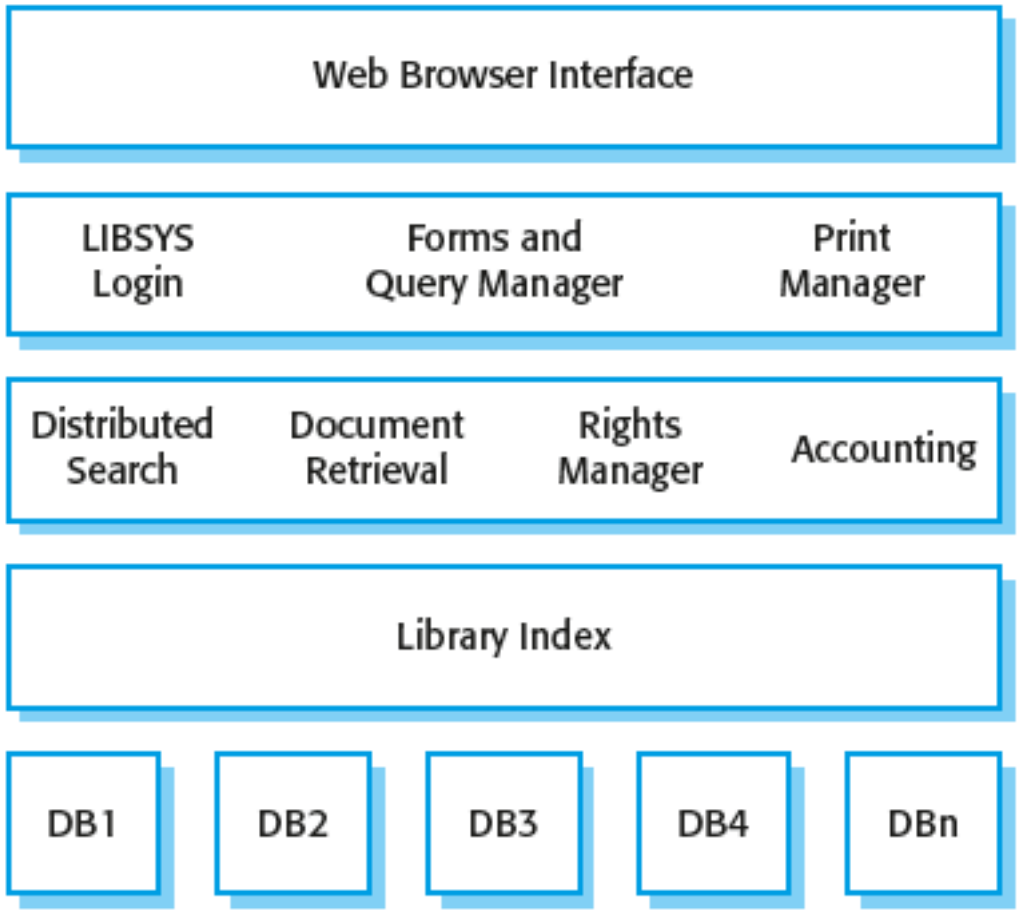
- 階層架構模式(Layered Architecture Pattern)
  - 子系統/元件/模組之間區分成不同的階層。
  - 每一階層都會開放一組嚴謹定義的介面(Well-Defined Interface)，讓上層元件利用。



https://bytebytego.com/guides/6-software-architectural-patterns-you-must-know/

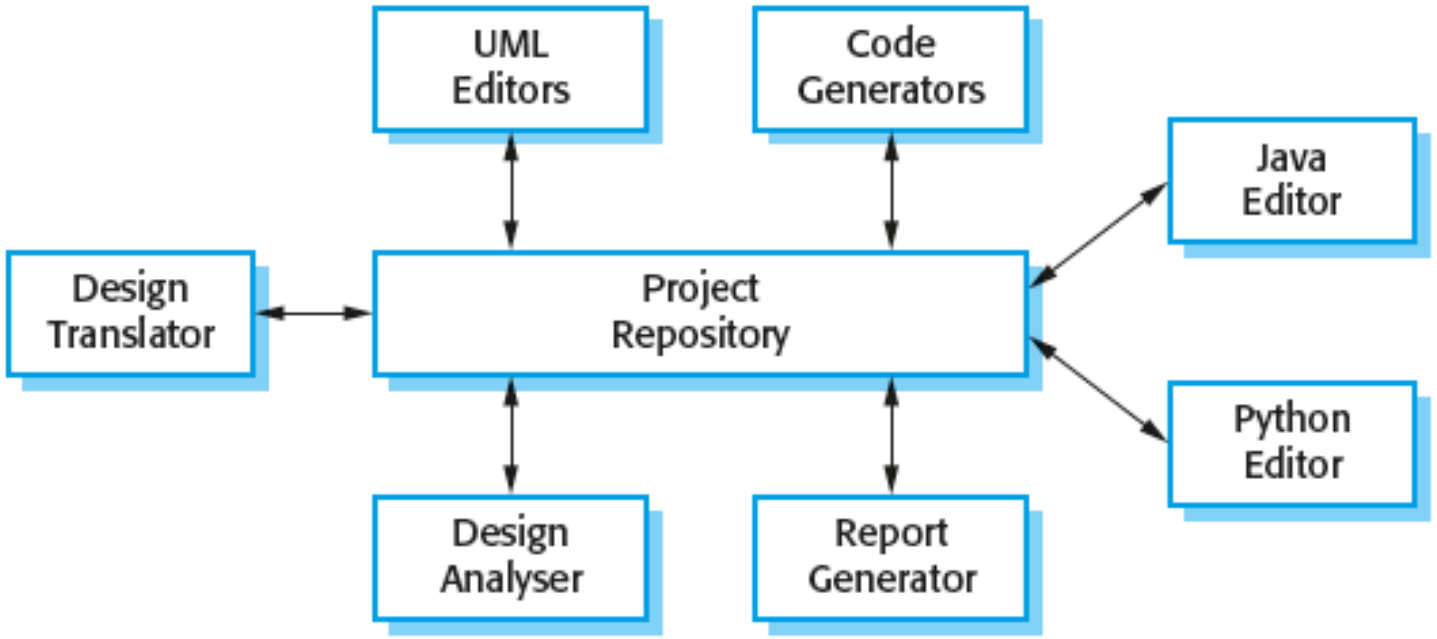
架構模式-階層架構模式2

- 階層架構案例：圖書館系統



架構模式 - 儲存庫架構模式

- 儲存庫架構(Repository Architecture)
  - 一群相互溝通的模組共用資料庫
- 儲存庫架構案例：CodeGen IDE

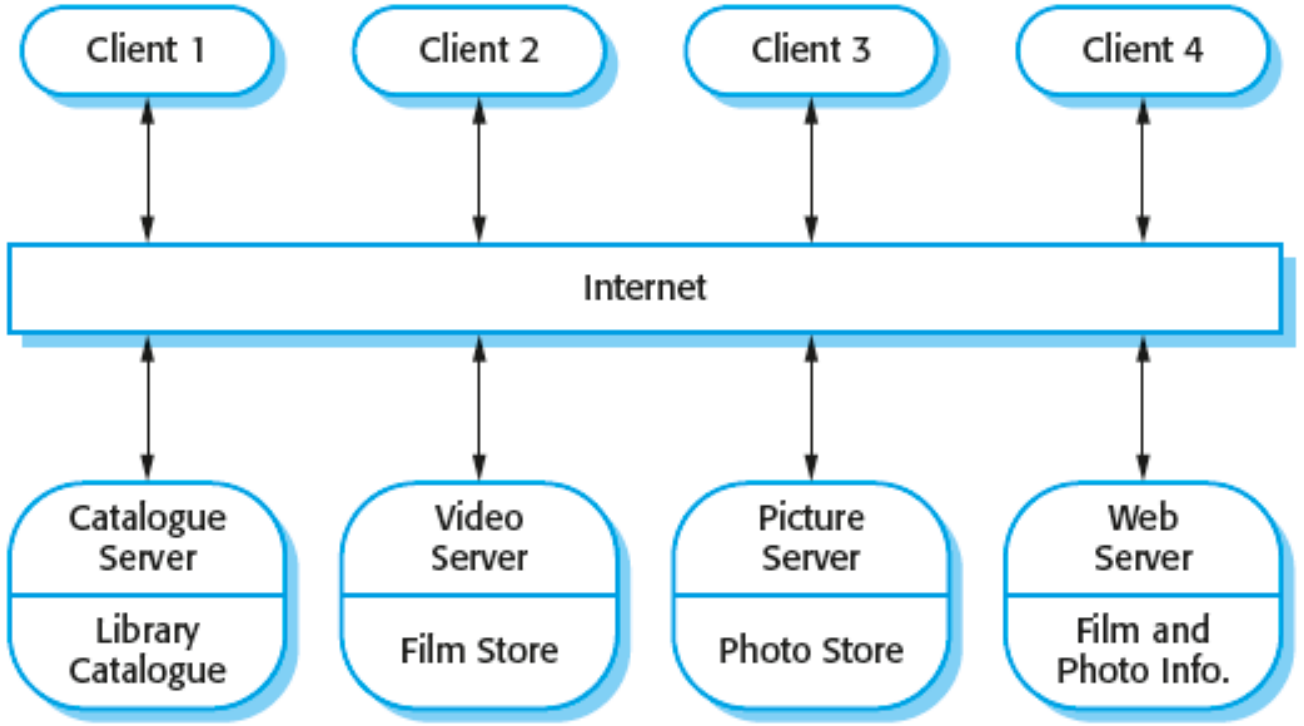


架構模式 - 分散式架構模式1

- 分散式架構模式(Distributed Architecture Pattern)
  - 架構在網際網路環境中，其系統各個模組分別部署於網路上多台主機，以提升系統的可擴充性及運算能力。
  - 主從式架構(Client-Server Architecture)
    - 至少有一個子系統提供相關的服務（稱之為伺服器端），而數個客戶端子系統可同時存取伺服器端所提供的服務。
  - 多層式架構(N-Tier或稱Multi-Tier Architecture)
    - 軟體系統的各功能模組間有既定之"客戶-伺服器"關係，例如三層架構包含使用介面呈現模組、系統邏輯運算模組、資料處理模組，前者為後者之客戶端，後者為前者的伺服器端。
  - 點對點架構(Peer-to-Peer Architecture)
    - 沒有嚴格的客戶端與伺服器端的限制，亦即，一個模組可以是提供服務的伺服器端，同時又是要求服務的客戶端。

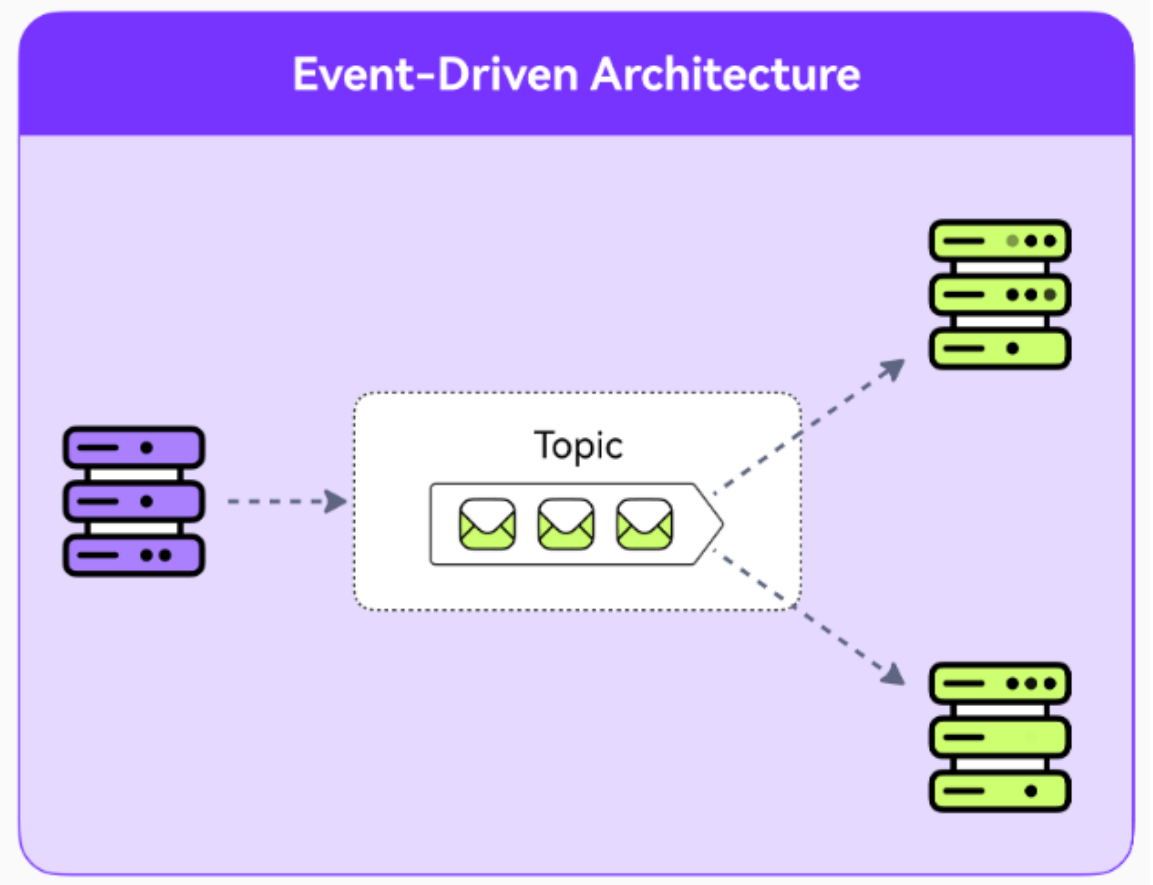
架構模式 - 分散式架構模式2

- 主從式架構案例: 多媒體分享系統



架構模式 - 分散式架構模式3

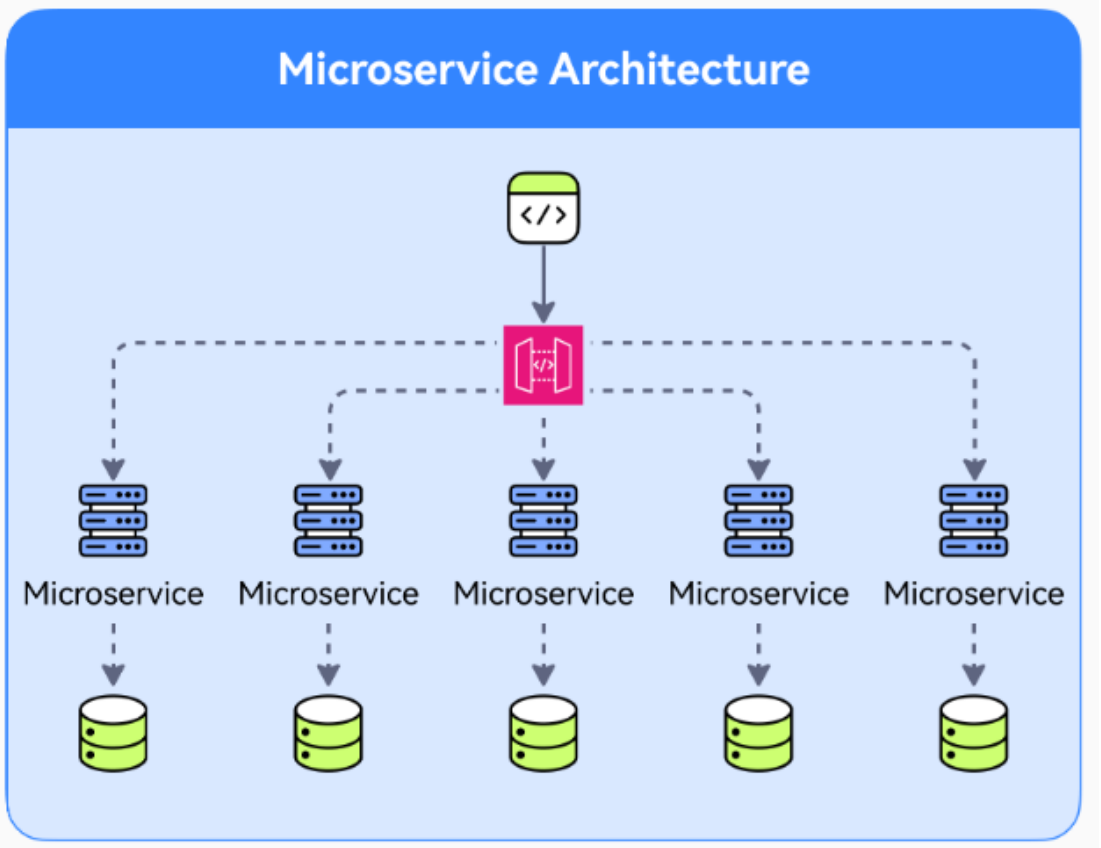
- 事件驅動架構(Event-Driven Architecture)
  - 服務透過發布其他服務可能會使用(或可能不會使用)的事件(Event)，以及訂閱其他服務發布的事件，來相互溝通。



https://bytebytego.com/guides/6-software-architectural-patterns-you-must-know/

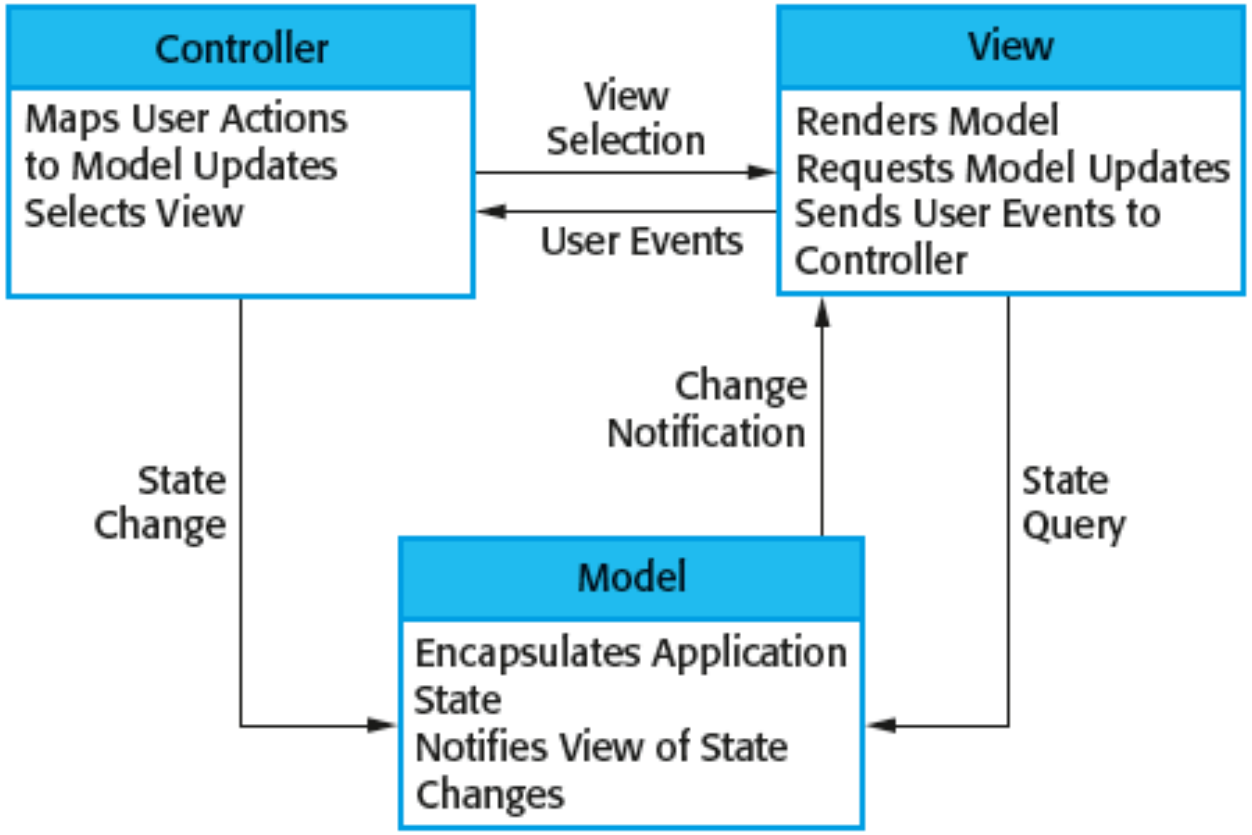
架構模式 - 分散式架構模式4

- 微服務架構(Microservice Architecture)
  - 將大型系統分解為更小、更易於管理的元件，稱之為微服務。
  - 微服務通常以容器技術運行。
  - 微服務間以API或Message溝通。
  - 微服務建議有獨立的Database。

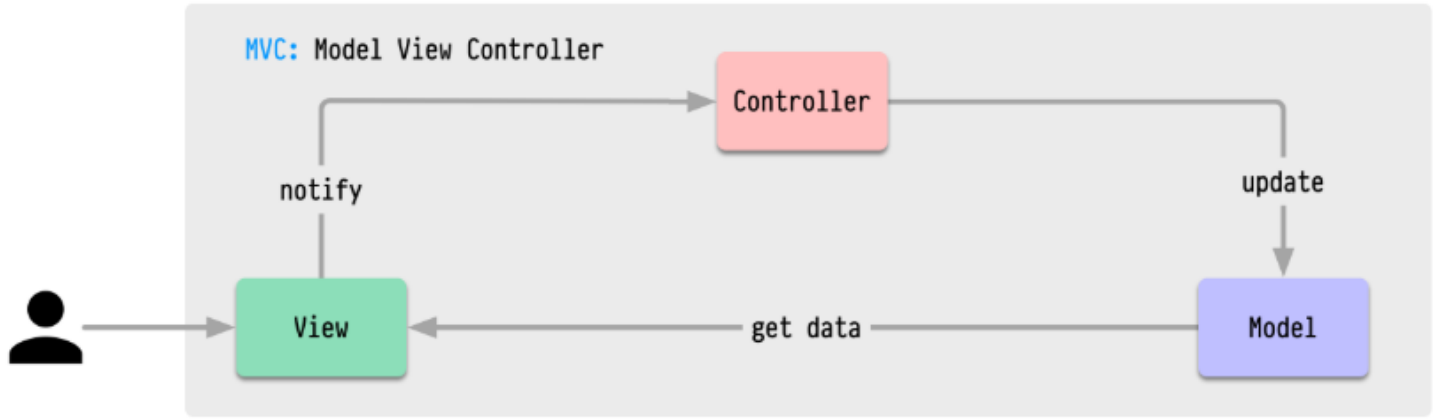


https://bytebytego.com/guides/6-software-architectural-patterns-you-must-know/

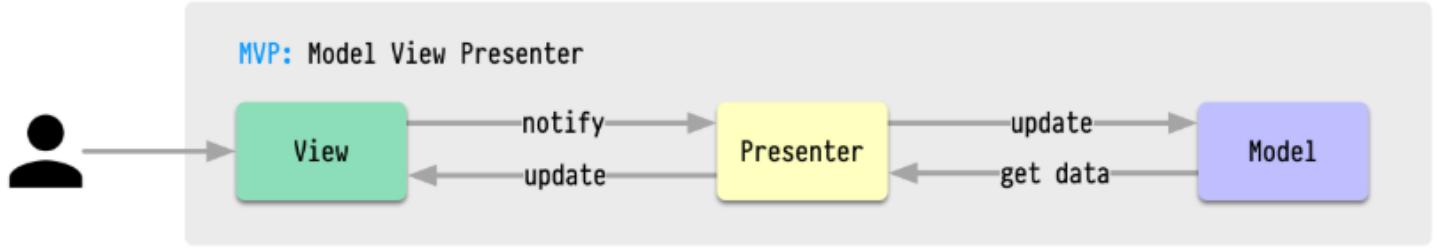
架構模式 - MVC架構模式



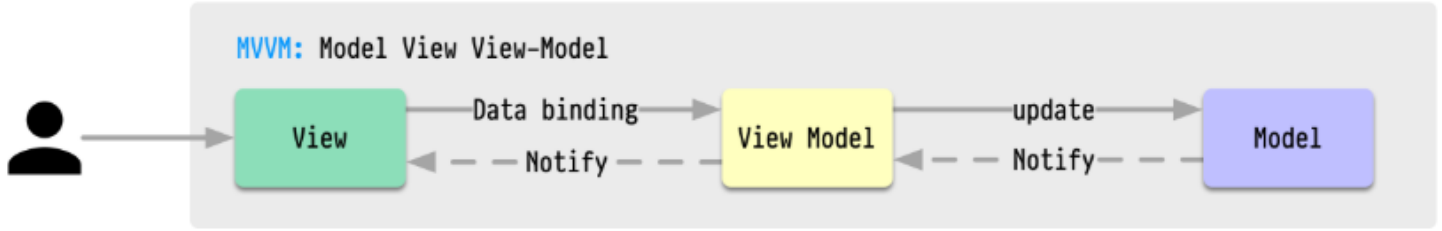
架構模式 – MVC, MVP, and MVVM



- Spring MVC
- PHP Laravel
- Django



- Android App
- JavaFX

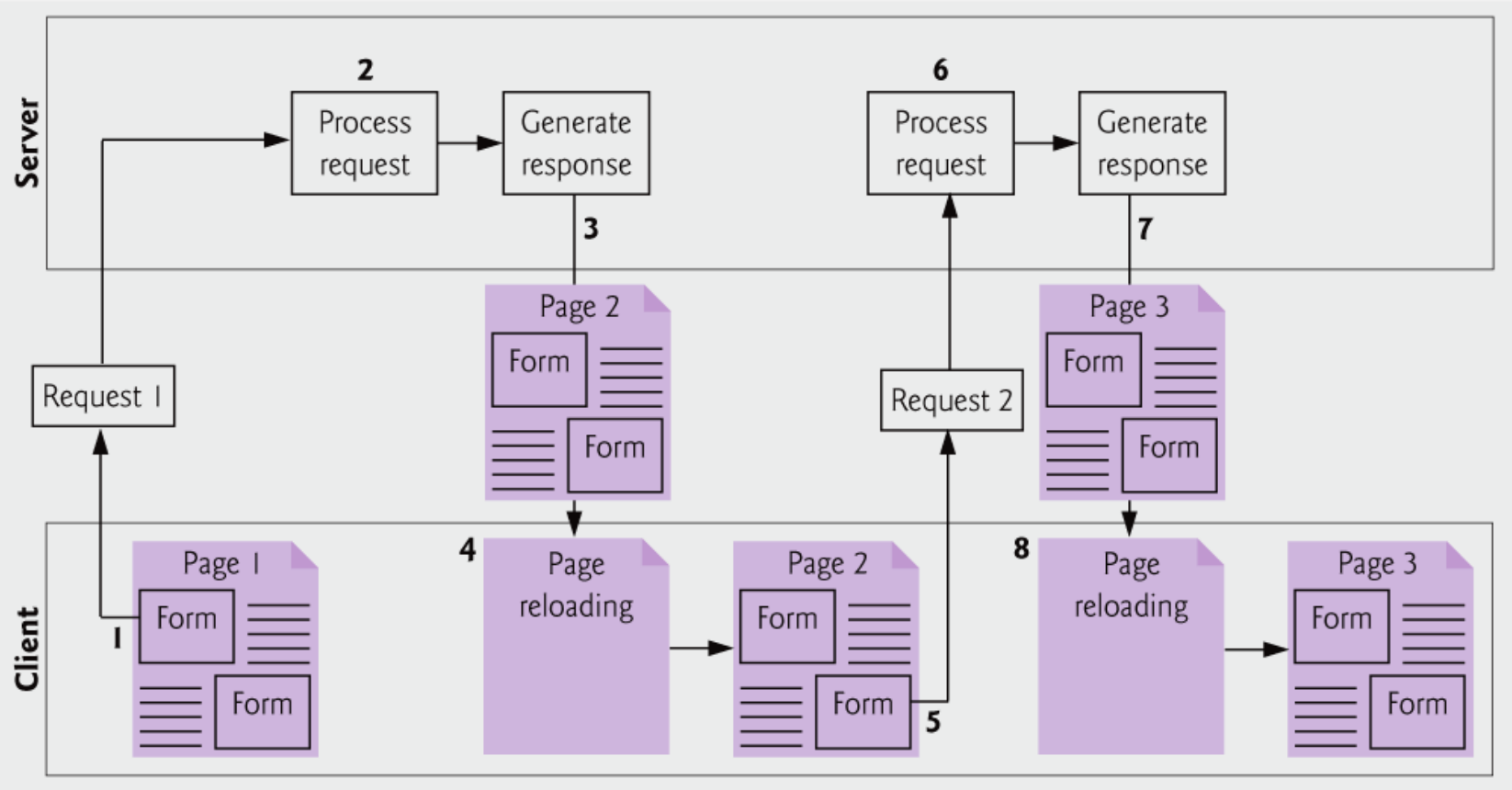


- iOS SwiftUI
- Vue.js

https://bytebytego.com/guides/mvc-mvp-mvvm-viper-patterns/

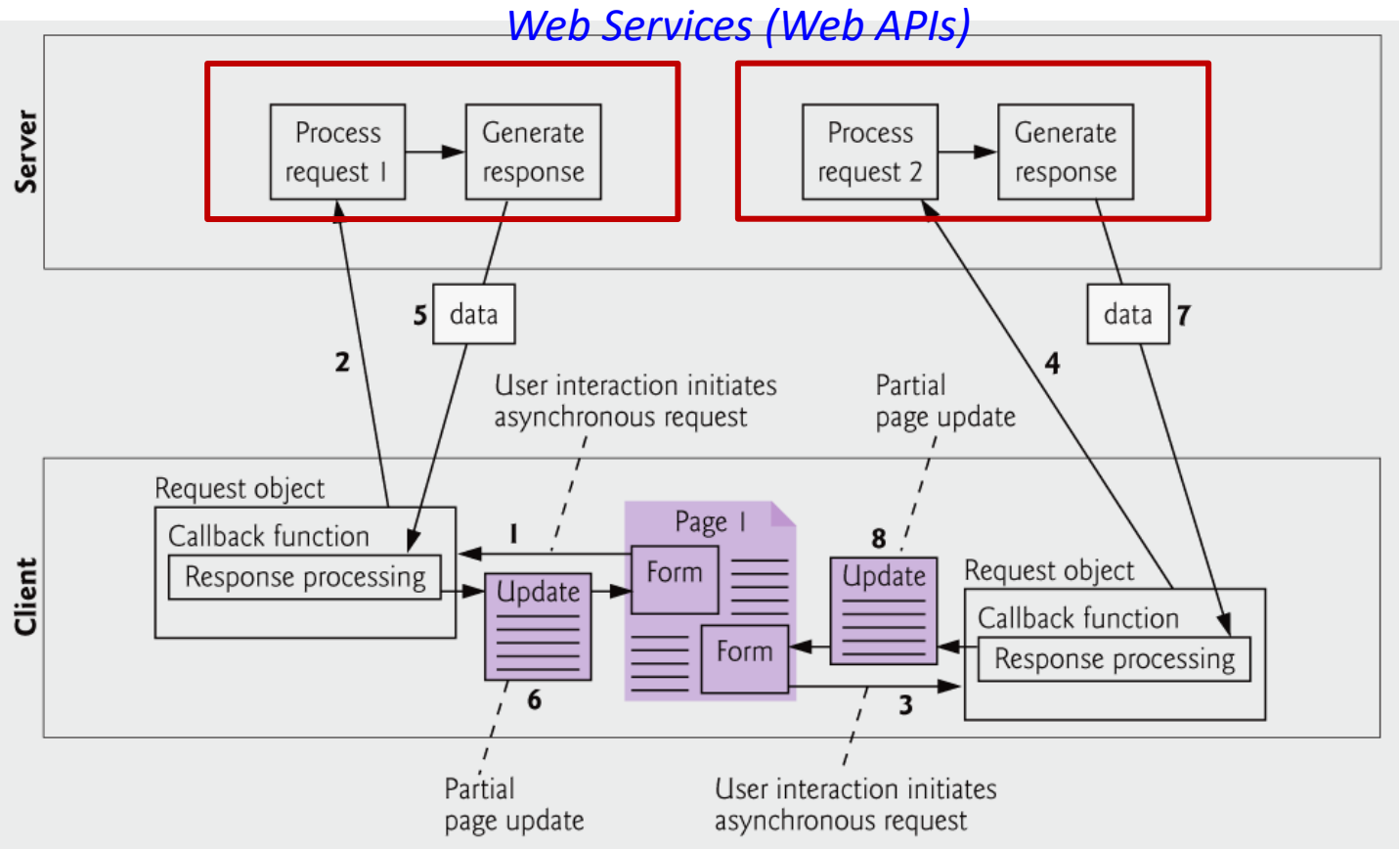
架構模式 - Web前後端混合架構

- Web前後端混合架構



架構模式 - Web前後端分離架構

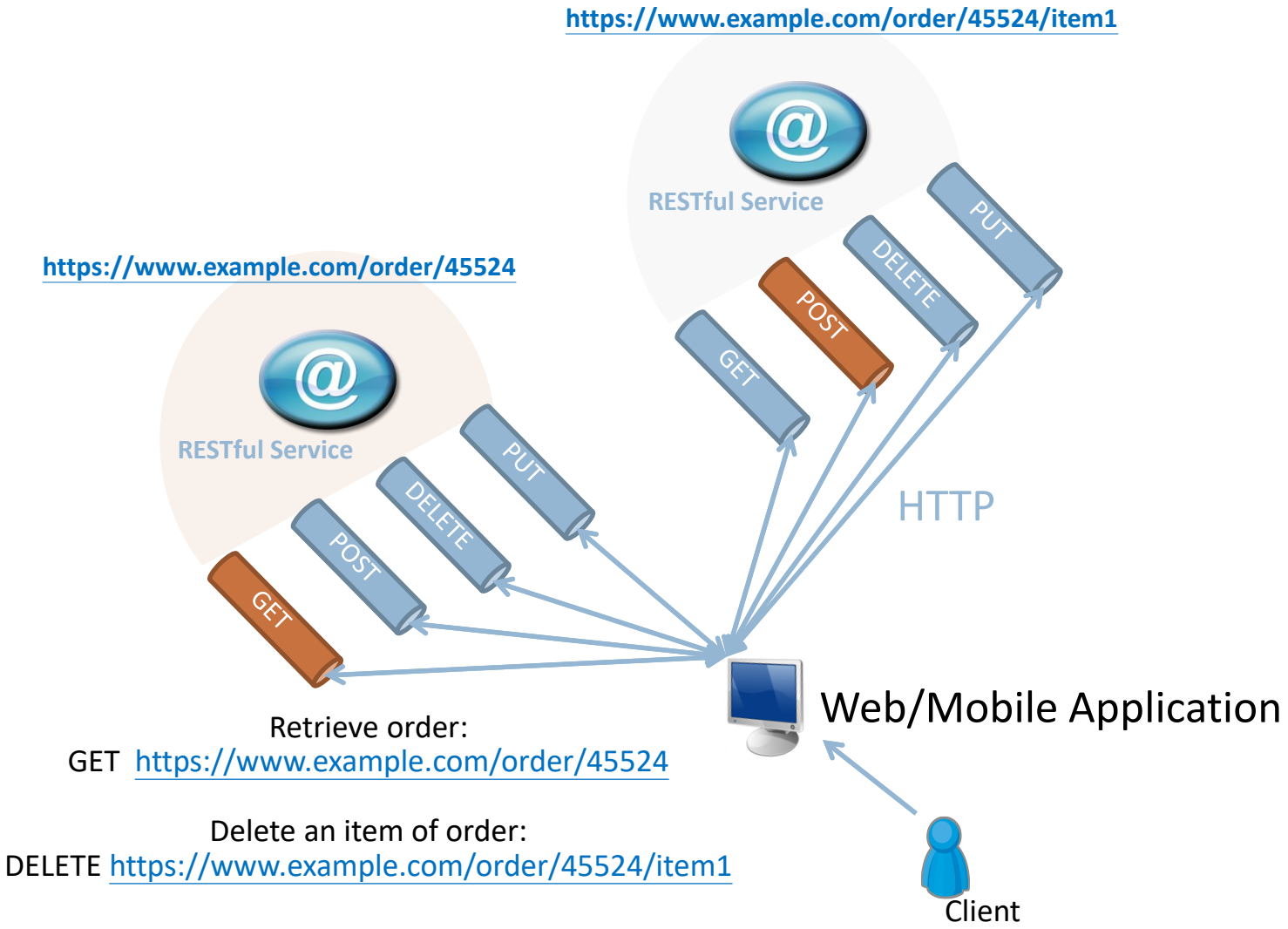
- Web前後端分離架構



API架構模式 - REST

- API: Application Programming Interface
- REST (REpresentational State Transfer)
  - Functionality are abstracted into resources.
  - Every resource is **uniquely addressable** using a universal syntax for use in hypermedia links.
  - All resources share a **uniform interface**
    - POST, GET, PUT/PATCH, DELETE → Create, Read, Update, Delete (CRUD)

RESTful Service/API Interface

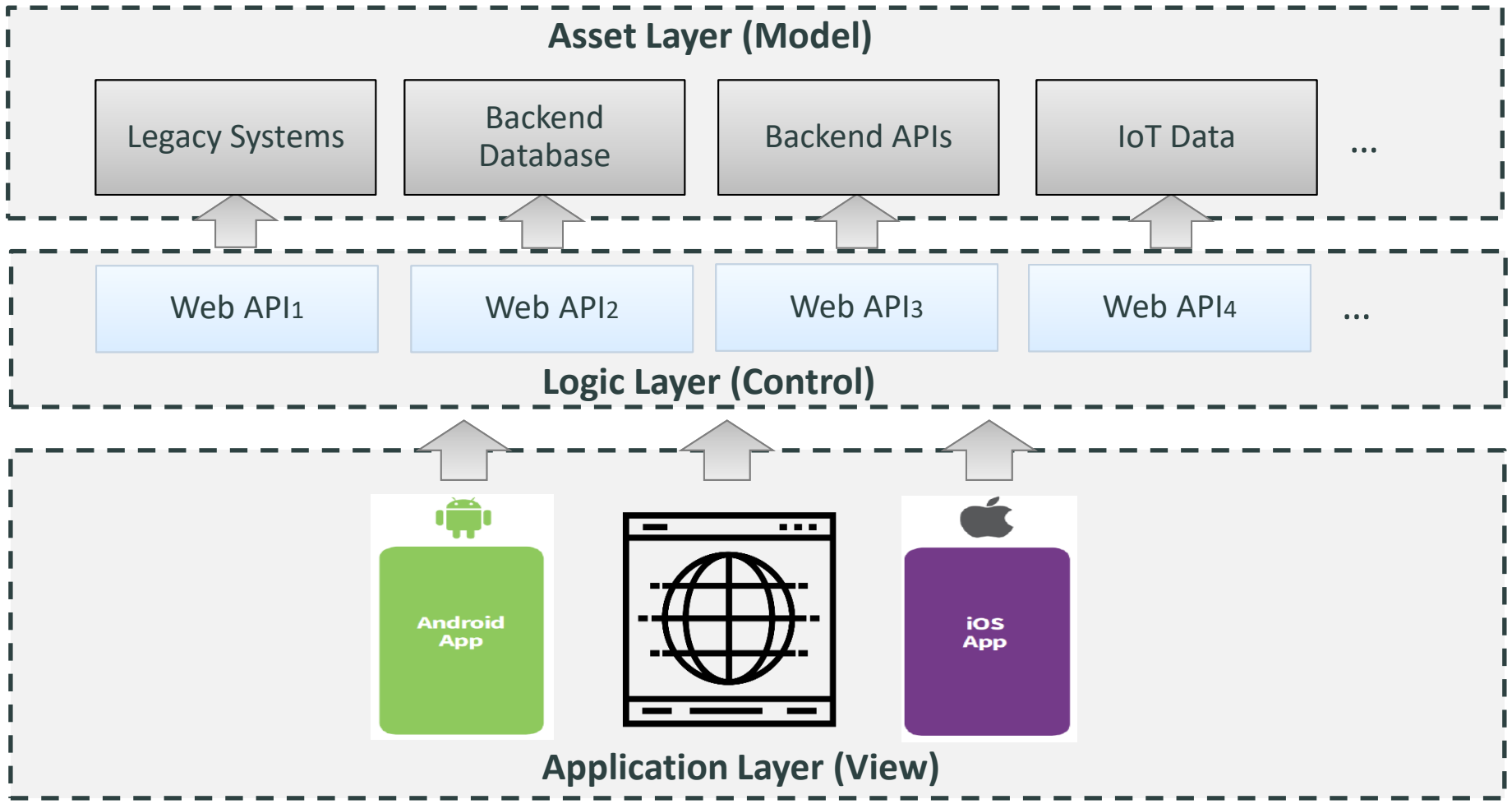


Retrieve order:
GET https://www.example.com/order/45524

Delete an item of order:
DELETE https://www.example.com/order/45524/item1

Real Example:
https://jsonplaceholder.typicode.com/users
https://jsonplaceholder.typicode.com/users/2

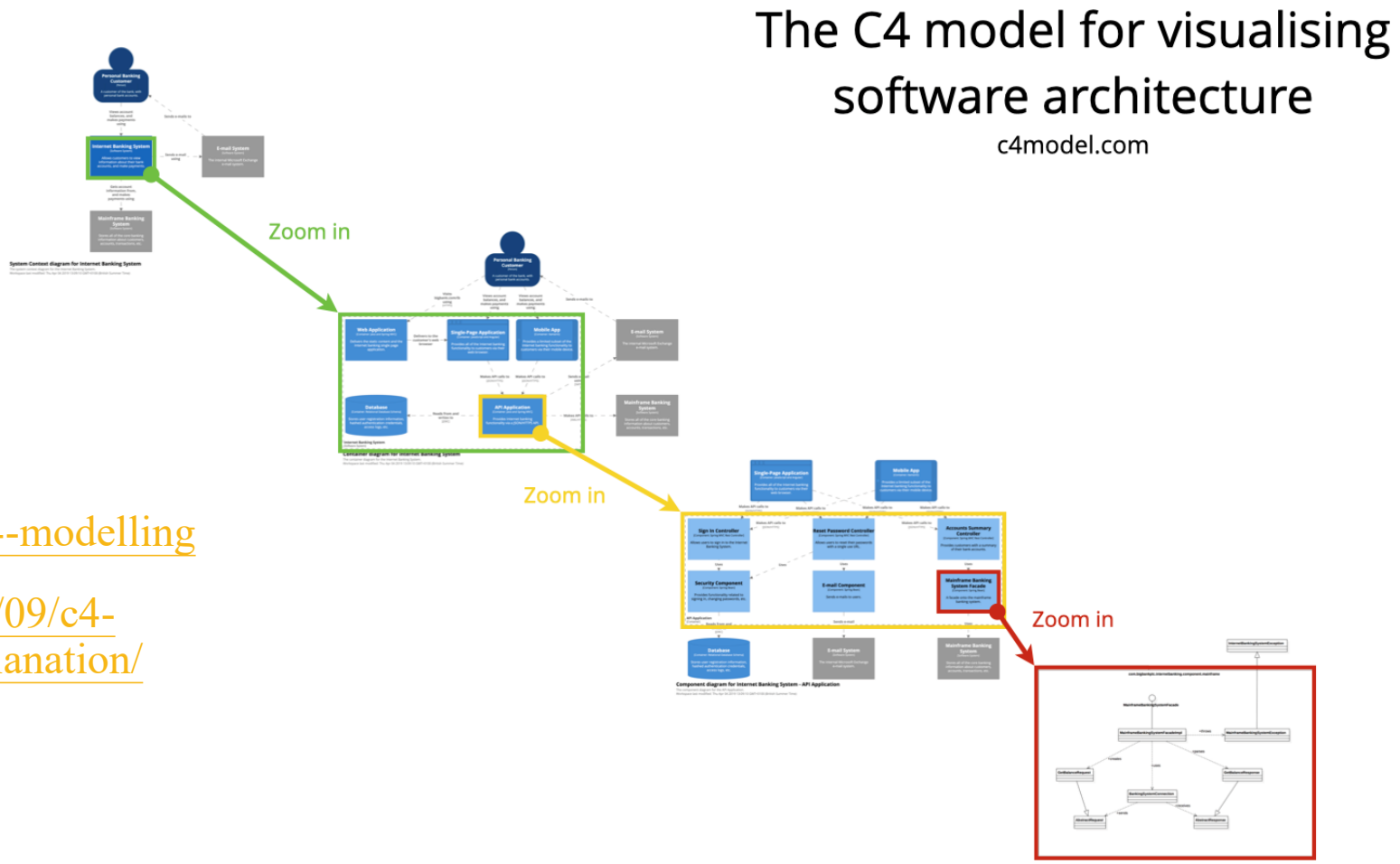
Web/Mobile Application: 典型架構範例



Web/Mobile Application: 可能實作方案

- (1) 前端程式：
  - HTML + CSS + JS (jQuery/Vue/React/Angular) + Bootstrap (RWD)
  - Android (Java/Kotlin) / iOS (Swift)
- (2) 後端程式：
  - Node.js (Express)
  - Java Spring/Spring Boot
  - PHP (Laravel)
  - Python (Flask/Django)
- (3) Database:
  - MongoDB
  - MariaDB/MySQL
  - Google Firebase (連同後端程式都支援)

C4 Model



https://c4model.com/

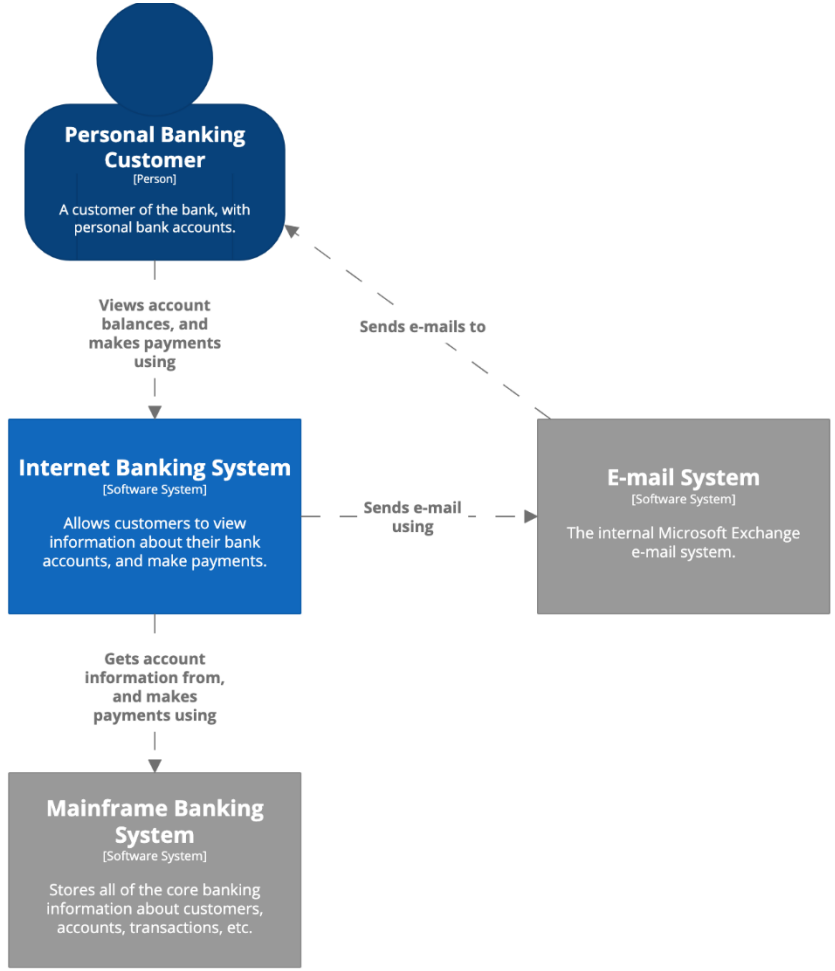
https://www.drawio.com/blog/c4-modelling

https://renierbotha.com/2024/06/09/c4-architecture-model-detailed-explanation/

C4 Model-Level 1: System Context Diagram

Detail isn't important here as this is your zoomed out view showing a big picture of the system landscape.

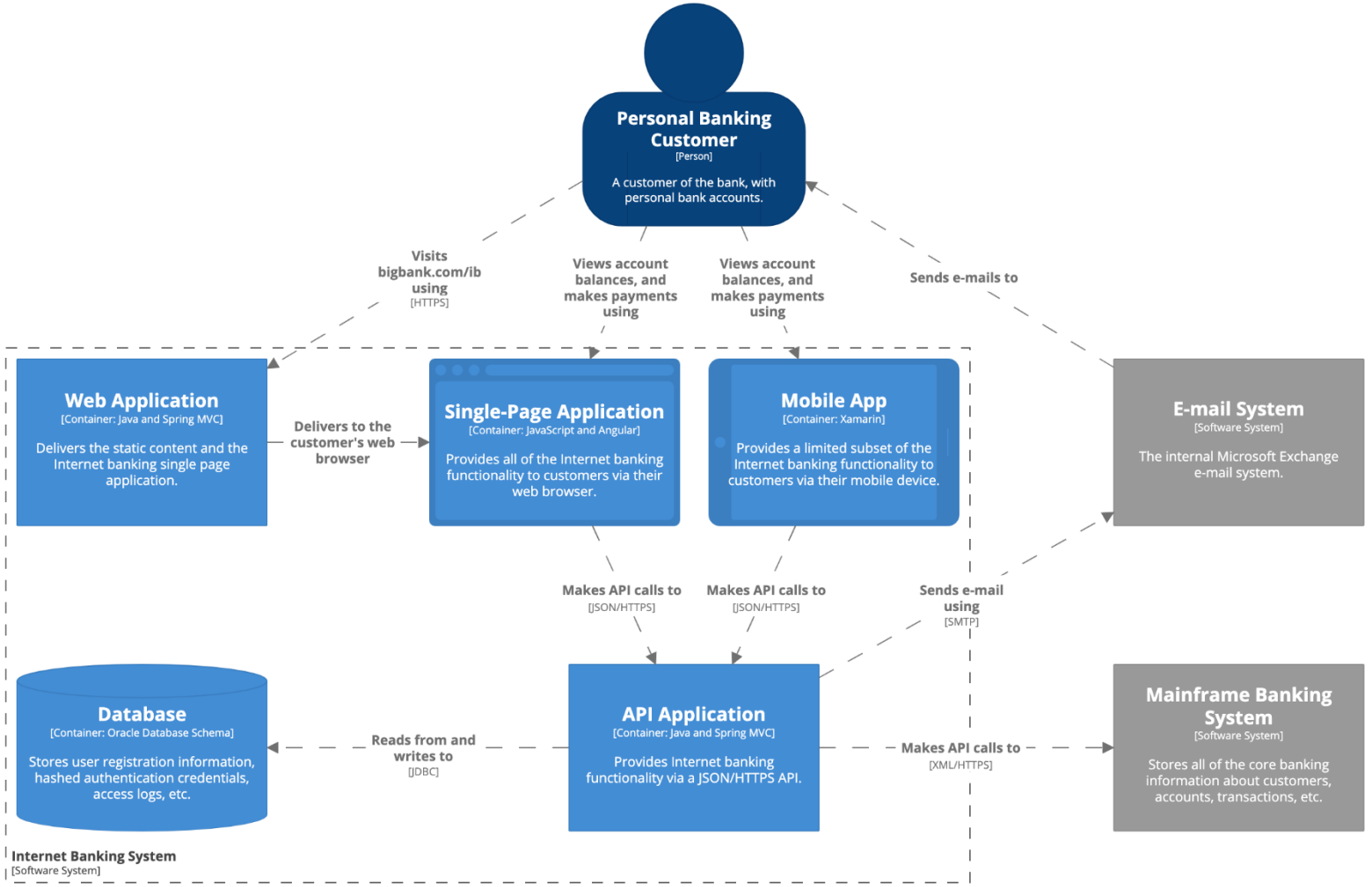
The focus should be on people (actors, roles, personas, etc) and software systems rather than technologies, protocols and other low-level details.



System Context diagram for Internet Banking System

The system context diagram for the Internet Banking System.
Workspace last modified: Wed Feb 05 2020 09:33:36 GMT+0100 (Central European Standard Time)

C4 Model-Level 2: Container Diagram[1]



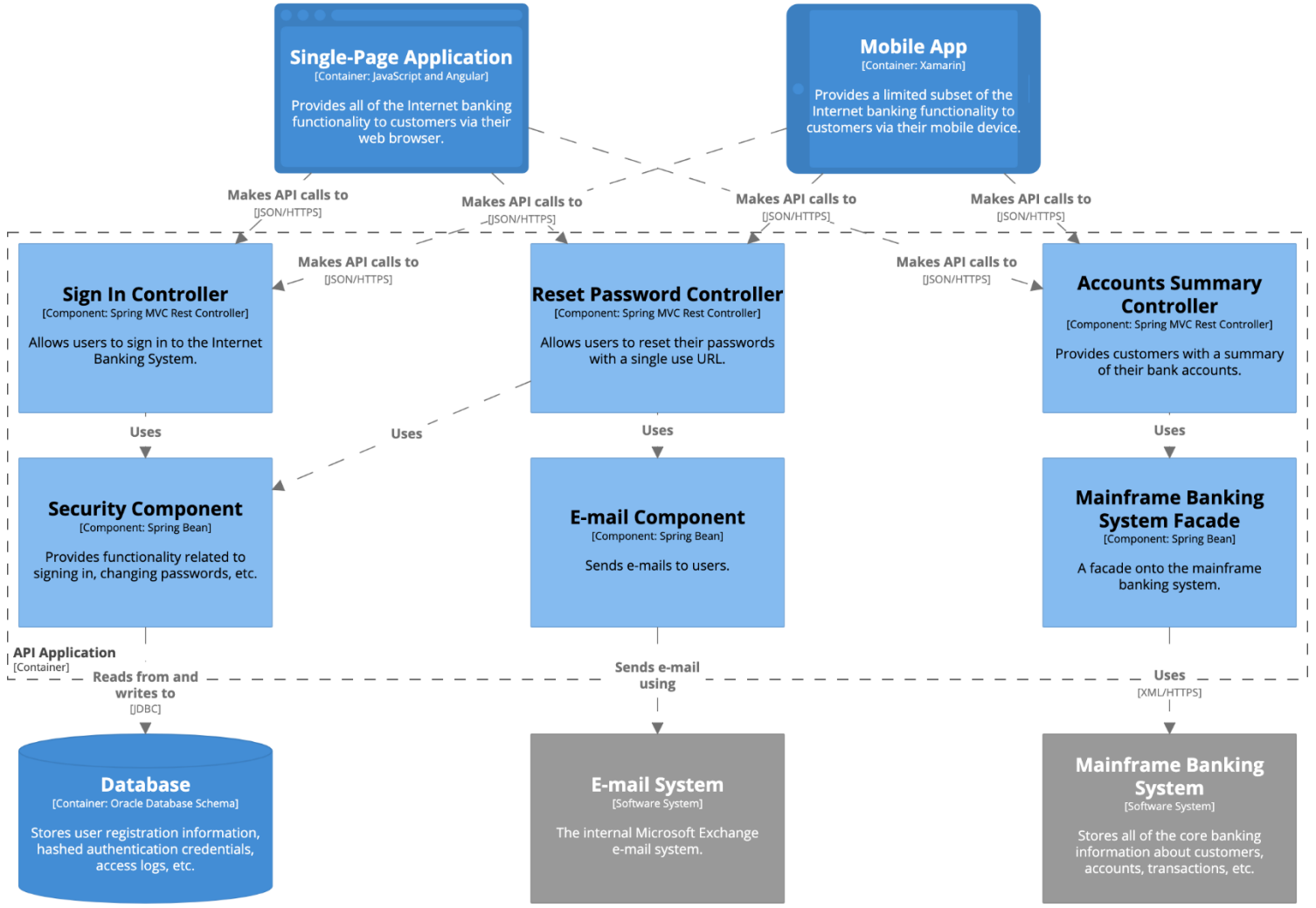
Container diagram for Internet Banking System

The container diagram for the Internet Banking System.
Workspace last modified: Wed Feb 05 2020 09:33:36 GMT+0100 (Central European Standard Time)

C4 Model-Level 2: Container Diagram2

- In the C4 model, a container represents an application or a data store.
  - Not Only Docker!
  - 彼此之間通過明確定義的介面
  - 通常運行在自己的Process或環境中
  - 每個Container都有特定的技術和責任範圍
- The Container diagram shows the high-level shape of the software architecture and how responsibilities are distributed across it.
- It also shows the major technology choices and how the containers communicate with one another.
- It's a simple, high-level technology focussed diagram that is useful for software developers and support/operations staff alike.

C4 Model-Level 3: Component Diagram[1]



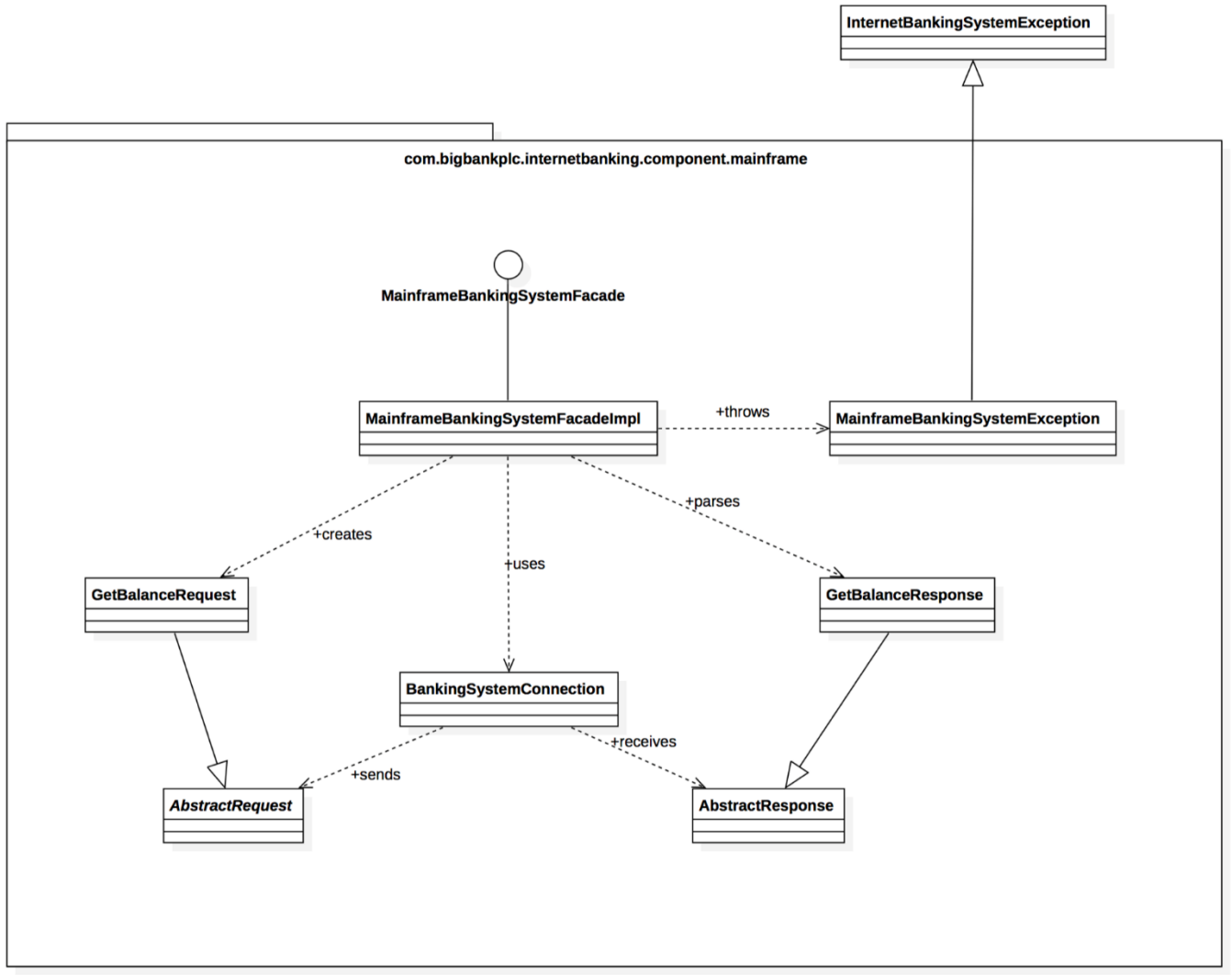
Component diagram for Internet Banking System - API Application

The component diagram for the API Application.
Workspace last modified: Wed Feb 05 2020 09:33:36 GMT+0100 (Central European Standard Time)

C4 Model-Level 3: Component Diagram2

- The Component diagram shows how a container is made up of a number of "components", what each of those components are, their responsibilities and the technology/implementation details.

C4 Model-Level 4: Code Diagram[1]



C4 Model-Level 4: Code Diagram2

- You can zoom in to each component to show how it is implemented as code; using **UML class diagrams**, **entity relationship diagrams (ERD)** or similar.
- You should consider showing only those attributes and methods that allow you to tell the story that you want to tell.
  - This level of detail is not recommended for anything but the most important or complex components.

物件導向技術與UML

- 物件導向系統分析是將問題空間分解成若干物件或概念。
  - 一個類別表示一群具有相同屬性、關聯、操作方法及相同意義的物件。
  - 物件導向的基本要件包含抽象、封裝、繼承、多型等。
- 分類描述任何事務必須使用適當的符號
  - 英國數學家Whitehead 認為：「優良適用的符號可讓人省卻不必要的工作腦力，集中在更上一層的問題，其效果將增進人類智能的力量」。
- UML (Unified Modeling Language)即提供一組圖形表示法來表達物件導向的概念
  - 舉例而言，類別用一個矩型表示，可以分為3 區塊，包括名字、屬性、操作方法。

如何瞭解及使用UML

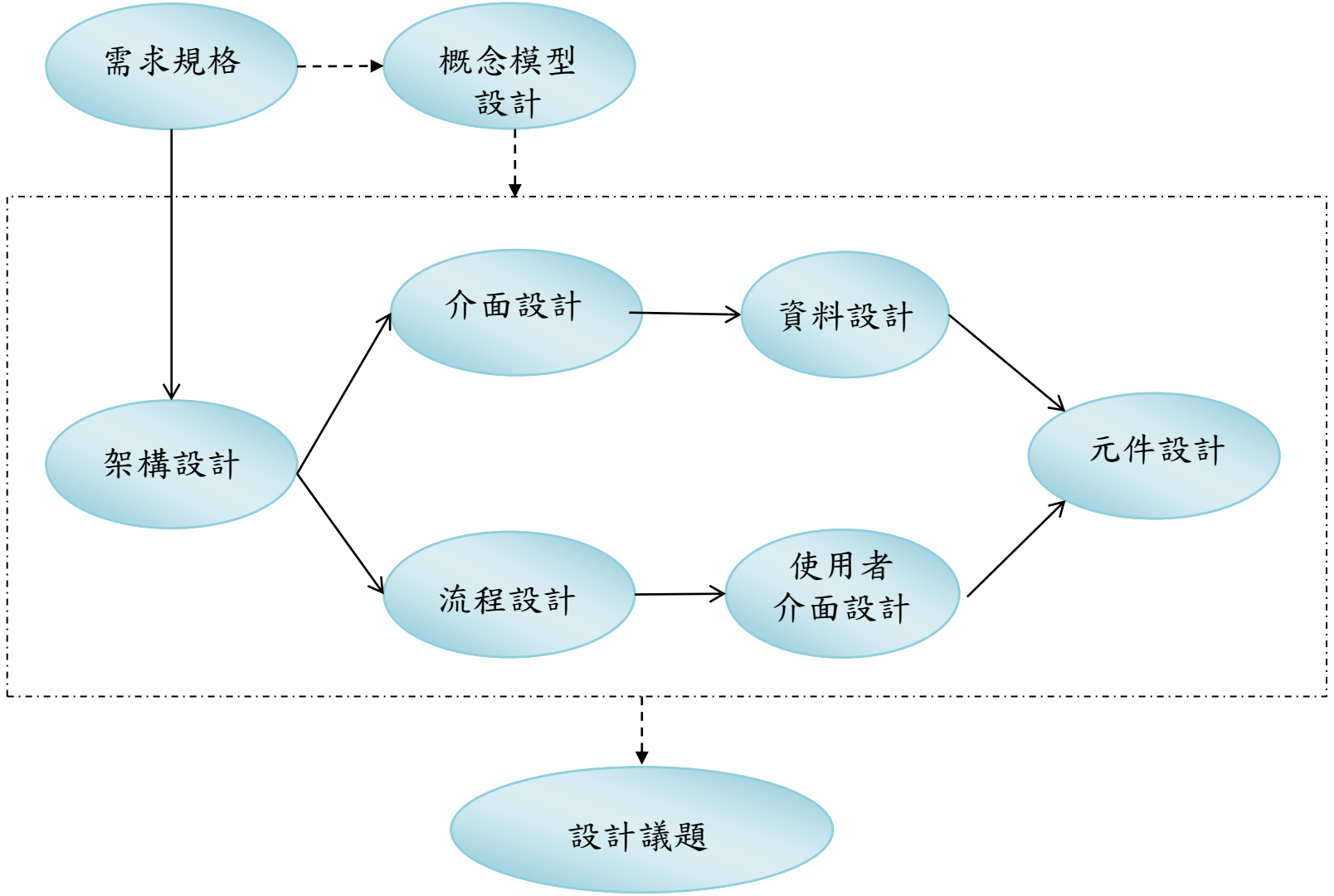
- 不需瞭解所有的UML圖
- 先寫下分析與設計內容，不須擔心圖示正確性
- 從工作中，逐漸探索及發現到關於系統外觀行為、內部結構等設計問題，並將之畫圖表達出來
- 再對UML圖及語法等進行一些研讀及瞭解，逐漸地將非標準的分析與設計圖轉成標準的UML圖
- 繪製UML圖的主要目的是為了釐清思考與幫助溝通

UML工具

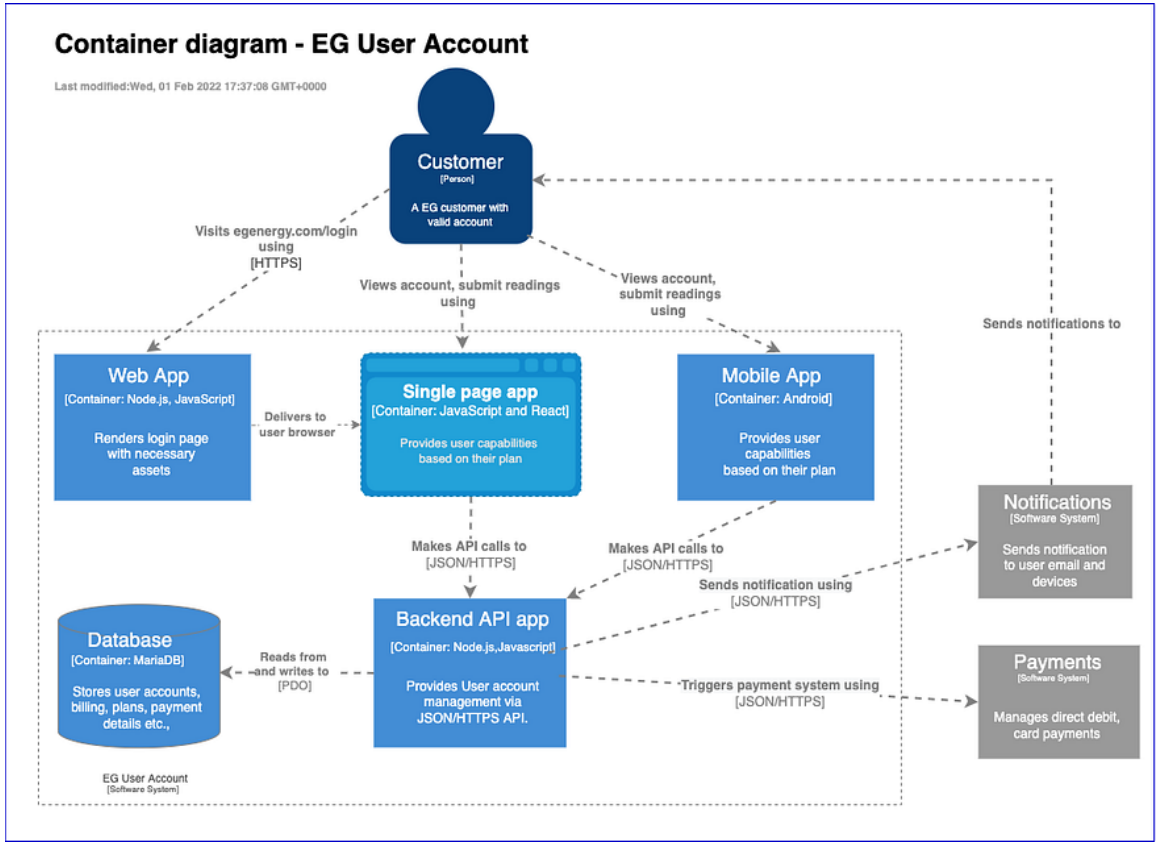
- StarUML
  - https://staruml.io/download/
- draw.io
  - https://app.diagrams.net/
- PlantUML
  - https://plantuml.com/zh/
- Mermaid
  - https://mermaid.js.org/
- Astah Community
  - https://astah.net/products/astah-community/
- MagicDraw UML
  - https://www.3ds.com/products/catia/no-magic/magicdraw
- Visual Paradigm
  - https://www.visual-paradigm.com/
- MS Visio
  - https://www.microsoft.com/zh-tw/microsoft-365/visio/flowchart-software

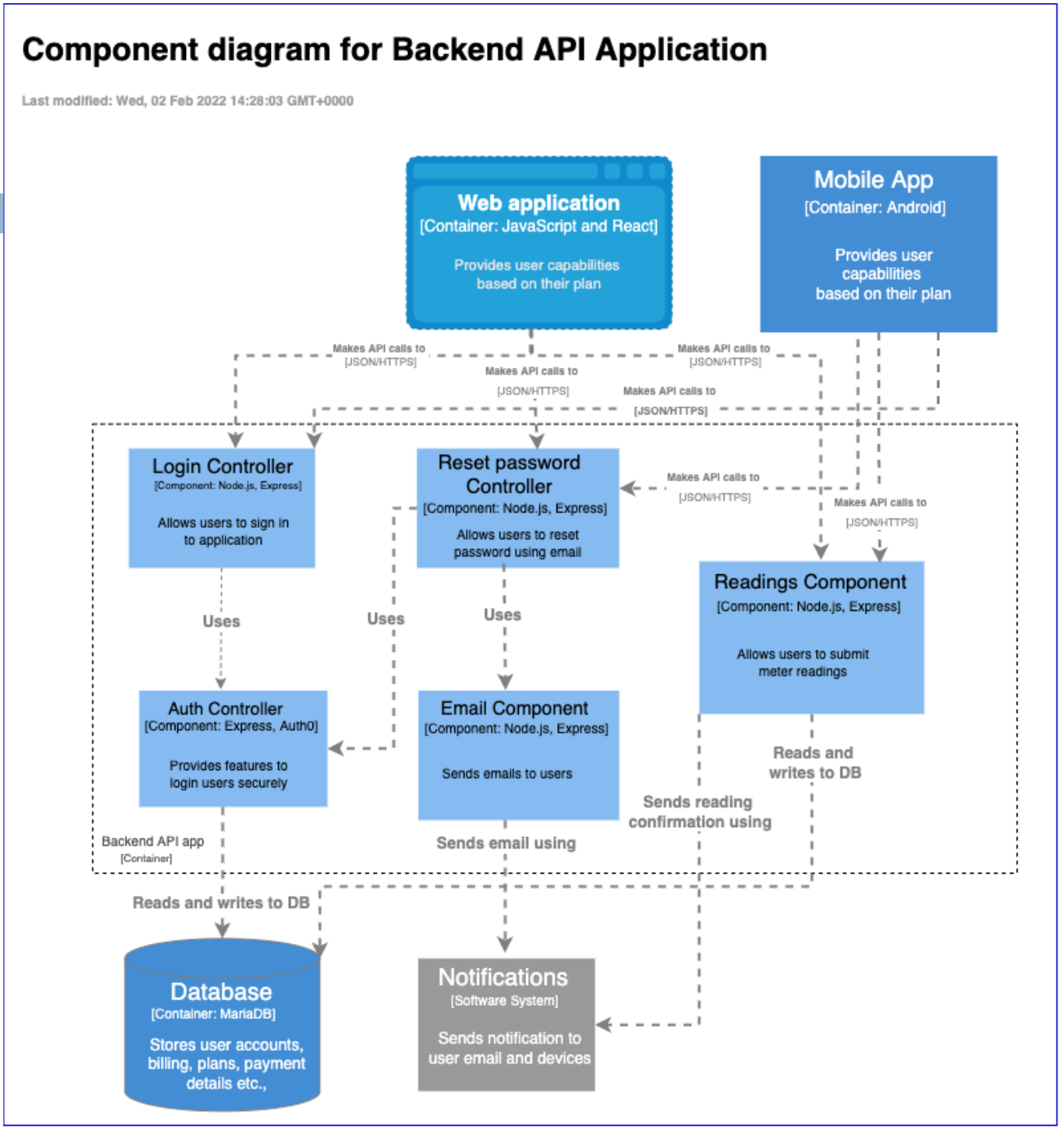
UML範例：https://creately.com/blog/diagrams/uml-diagram-types-examples/

軟體設計規劃：建議步驟



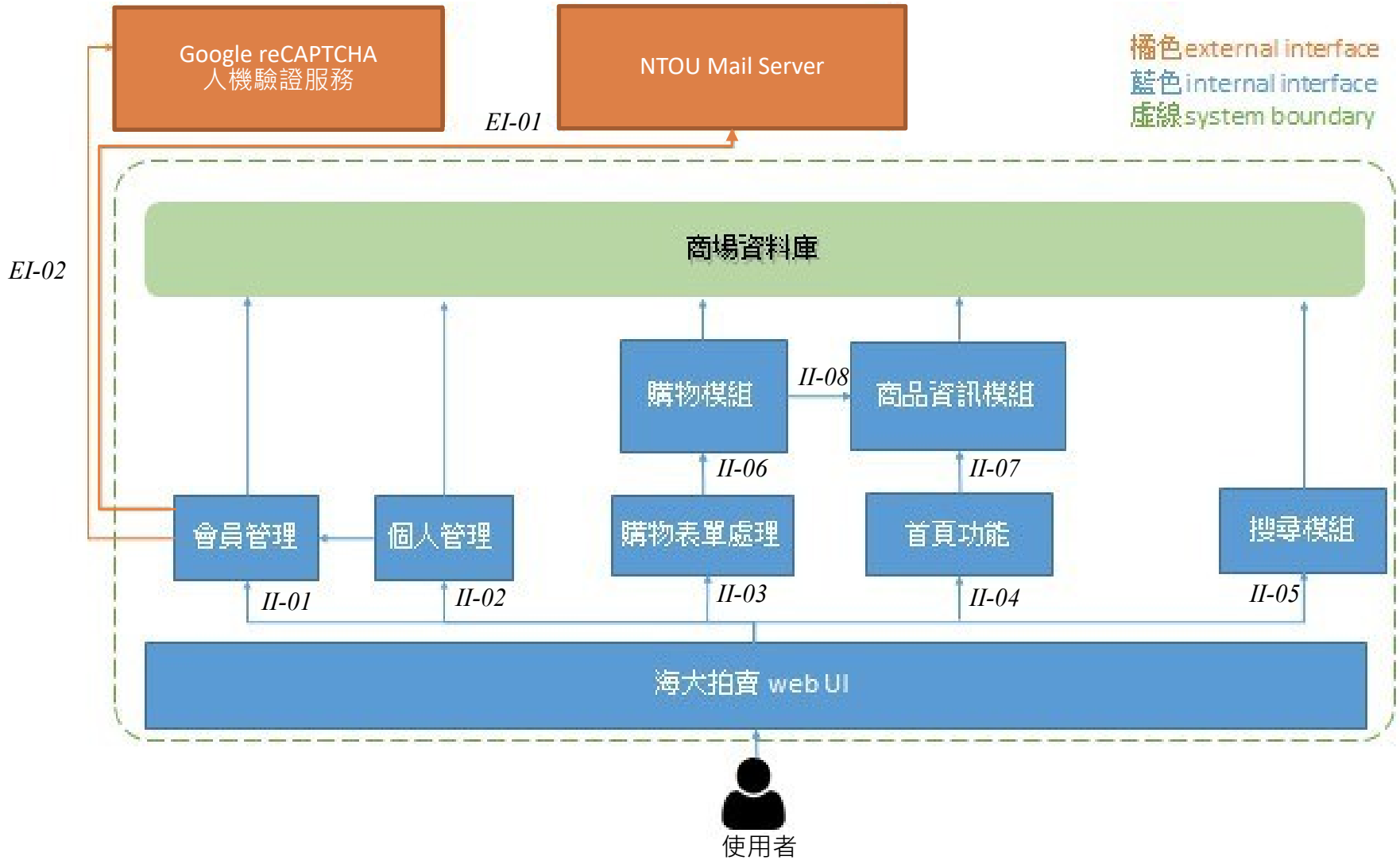
架構設計-範例1





https://medium.com/news-uk-technology/c4-model-a-better-way-to-visualise-software-architecture-df41e5ac57b8

架構設計-範例2


Google reCAPTCHA
人機驗證服務
NTOU Mail Server
EI-01
EI-02
橘色external interface
藍色internal interface
虛線system boundary
商場資料庫
購物模組
II-08
商品資訊模組
II-06
II-07
會員管理
個人管理
購物表單處理
首頁功能
搜尋模組
II-01
II-02
II-03
II-04
II-05
海大拍賣 web UI
使用者

介面設計

- 介面(Interface)不是專指User Interface，亦代表兩個軟體模組(或兩個系統)銜接時，一個模組要如何呼叫或嵌入另外一個模組
  - 舉例而言，若GUI模組要呼叫圖片管理模組，圖片管理模組開放給GUI模組的method就是介面
    - 介面可視為API規格(包含輸出入資料)
  - 介面規格需要在實作前就制定好，如此呼叫的模組與被呼叫的模組方可平行開發，屆時模組完成後(單元測試後)才能順利整合。

系統架構與介面設計

- 前端頁面可以合併為同一模組，或是拆更細。
- 後端API建議可以各自獨立成模組。
- 前端頁面與後端API之間的內部介面(Internal Interface)應該是介面設計的重點。
- 系統模組(前端頁面或後端API)與外部系統間的外部介面(External Interface)也是重點。
- API與資料庫之間的介面通常是SQL或CRUD指令，相對來說，DB Table/Schema內容更為重要。
  - 因此，若要描述這部分的介面沒問題，但也可先略過。

介面設計範例1

介面名稱 (Interface Name)	介面提供者 (Interface Provider)	介面使用者 (Interface Consumer)
II-05. 搜尋介面	搜尋模組	海大拍賣web UI
連結方式	**輸入資料(參數範例)**	**輸出資料(JSON範例)**
http GET URL: /product_searching	keyword=PS5&minPrice=8000&maxPrice=20000	[{ "ProductID":"P001", "SellerID":"S002", "state":True, "ProductName":"PlayStation 5遊戲主機", "ImageURL":"https://tinyurl.com/yxeuxyay", "Amount":10, "Price":15000, "Information":"不用多說", "Category":"3C產品", }, ...]
對應之介面描述		
系統可由外部輸入條件，並搜尋對應之商品，以取得搜尋結果。(keyword必填)		

介面設計範例2

介面名稱 (Interface Name)	**介面提供者 (Interface Provider)**	**介面使用者 (Interface Consumer)**
EIR-002 路段車速與行車時間介面	車速與行車時間API	行車資訊UI模組
連結方式(Connection Type)	**輸入資料(Input Data)**	**輸出資料(Output Data)**

http POST	URI:
	https://[HOST IP]/RoaderServer/ServerJobsDispatcher.do
	
	Request (example):
	{
	"speedTimeRequest":[
	"SEC-0030-N-0230",
	"SEC-0010-N-0310"
	]
	}
		Response (example):
		[
		{
		"section":"SEC-0030-N-0230",
		"time":139,
		"speed":72
		},
		{
		"section":"SEC-0010-N-0310",
		"time":188,
		"speed":83
		}
		]

對應之介面描述(Interface Description)		
Client端可送出多組道路(roadID)與起訖里程至API，API會回傳相對應的路段車速與行車時間資訊。		

資料設計1

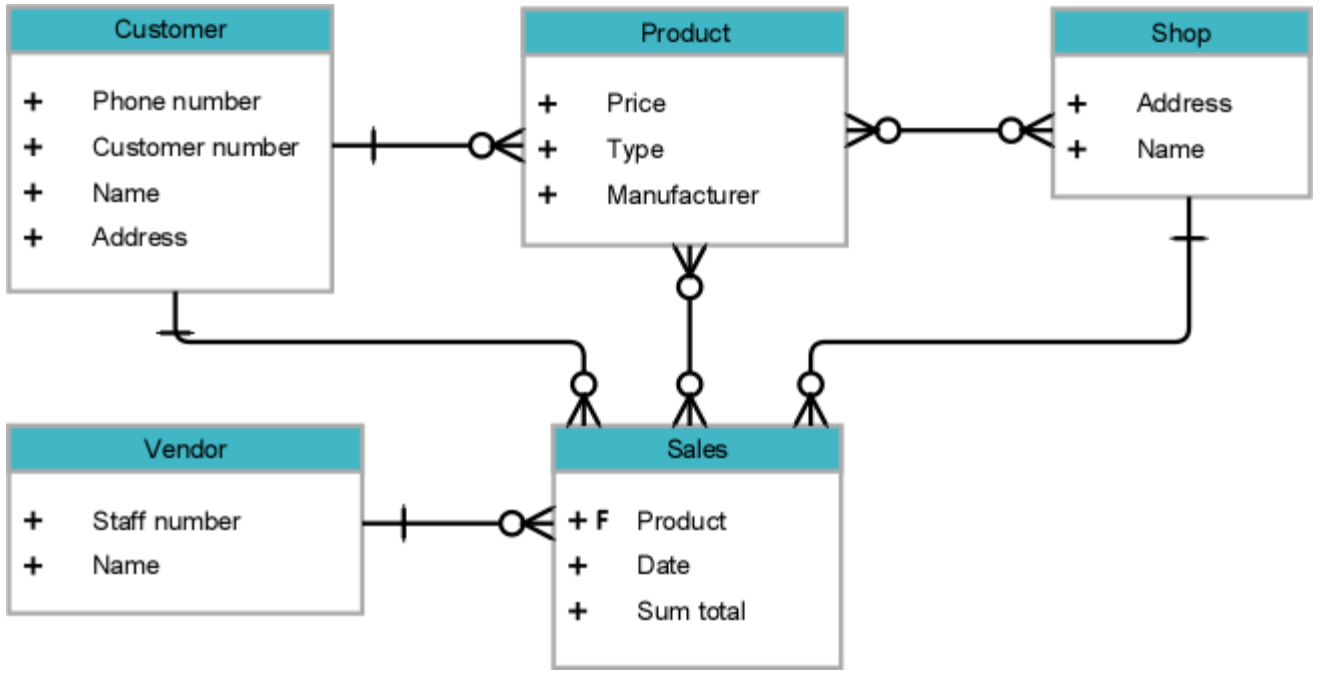
- 設計此系統會使用到的資料庫table schema、檔案結構、XML schema、JSON (JavaScript Object Notation)物件等資料結構。

資料庫table schema設計範例：

欄位名稱	欄位代號	型態	長度	備　註
帳號	user_id	varchar	50	primary key
密碼	password	varchar	80	Not Null
e-mail	email	varchar	100	Not Null
生日	birthday	date		Not Null
性別	sex	int	1	Not Null
居住地	address	varchar	100	Not Null
活動地區	area	varchar	35	Not Null
大頭照	user_photo	varchar	50	
最後登入日	last_login	date		
Facebook ID	FB_id	varchar	30	
暱稱	nickname	varchar	10	
臉書授權	accessToken	varchar	200	

資料設計2

- 可以用ERD (Entity-Relationship Diagram)去呈現資料庫設計



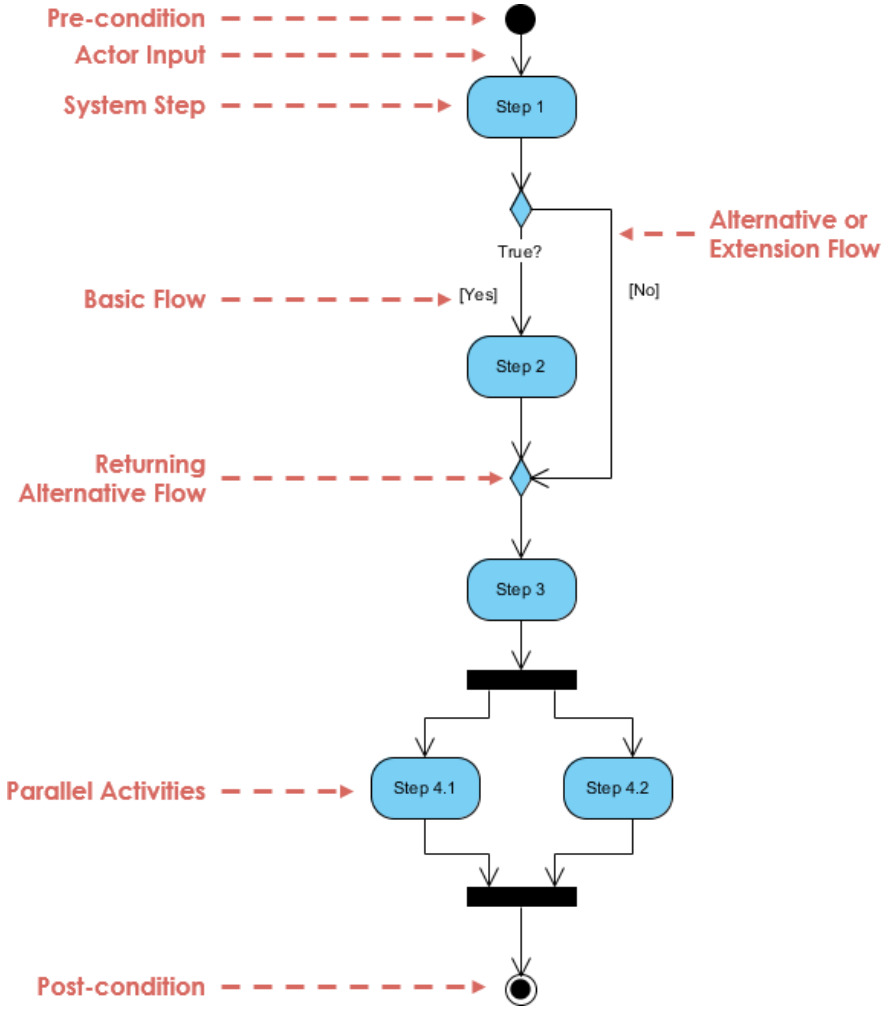
https://support.bizzdesign.com/display/knowledge/Modeling+with+ERD

流程設計1

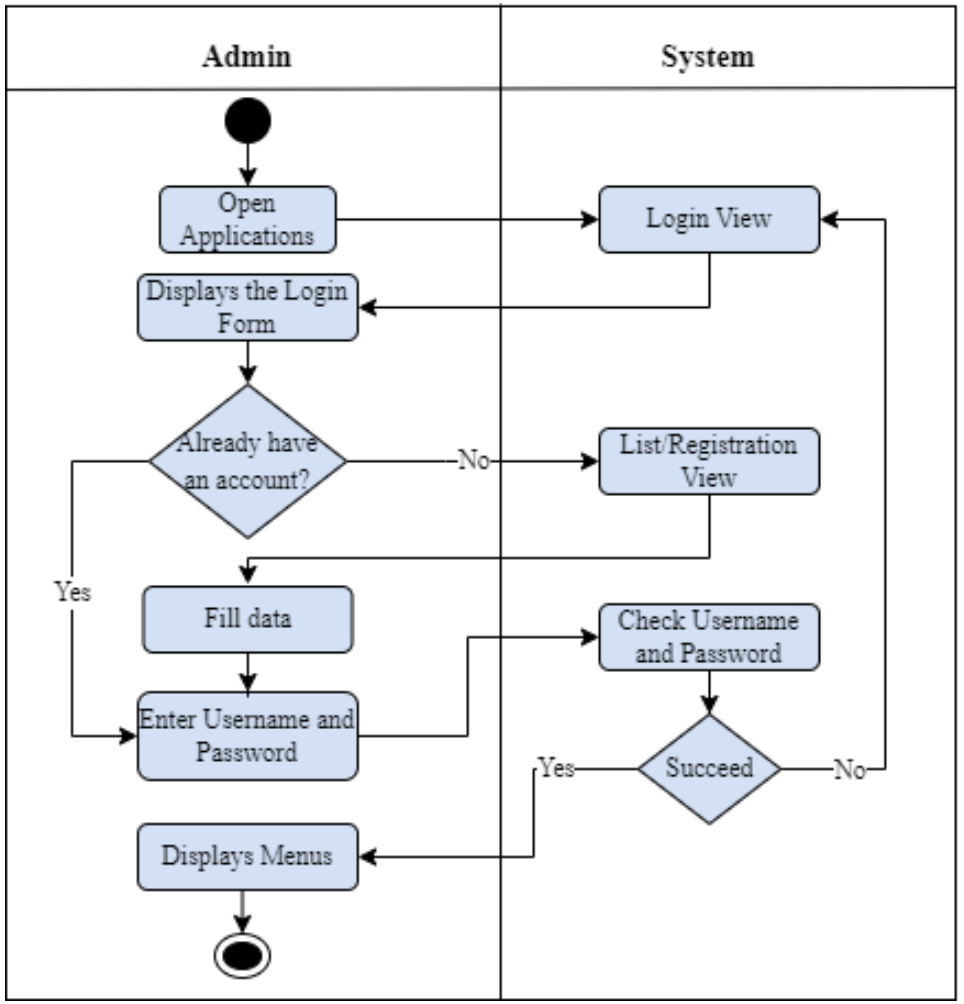
- UML活動圖(Activity Diagram)：描述所開發的系統流程，目的是設計整個系統的完整運作，應為操作概念的細部設計。
  - 描述業務流程
  - 表示系統操作流程
  - 展示並行活動
  - 說明決策分支
  - 模擬使用者操作流程

流程設計: Activity Diagram

Notation



Example



使用者介面(User Interface)設計

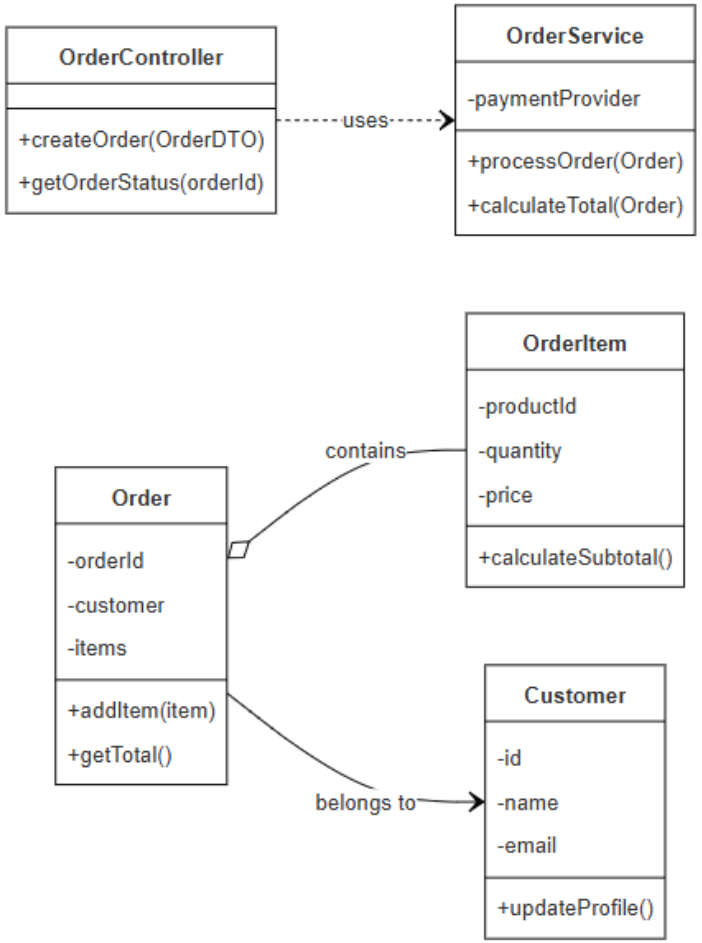
- 可用各種畫面設計方式，如發展網頁prototype、繪製PowerPoint投影片等，去設計你的主要系統畫面。
- 此部份應是SRS中使用者介面分析的進階設計版本(比需求階段更接近實際系統樣貌)。。
- 畫面設計可幫助團隊溝通系統的使用者介面呈現，並可及早確認使用者需求之落實情況。

元件設計

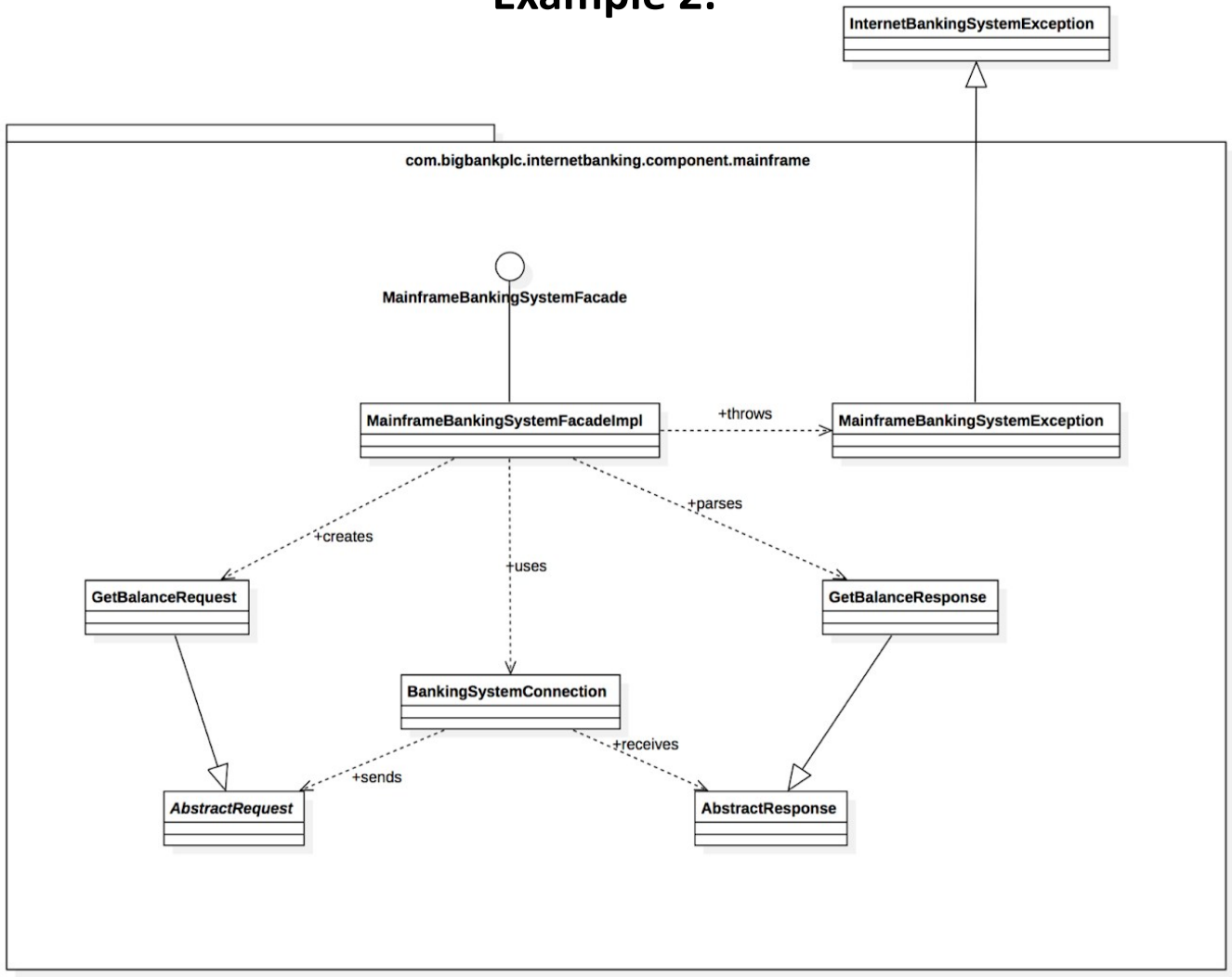
- 元件設計分為靜態面之結構設計與動態面之行為設計
  - 結構設計 (structure design)
    - UML類別圖(Class Diagram)：明確設計系統中包含哪些類別，以及這些類別之間的關係。若遇到非一般物件類別或特殊物件類別，如HTML、JavaScript、Servlet等，可用stereo type表達，如<<JavaScript>>、<<Servlet>>。
  - 行為設計 (behavior design)
    - UML循序圖(Sequence Diagram)：設計物件間的互動關係與循序執行。
      - https://docs.nomagic.com/display/MD184/The+formal+gate+and+actual+gate+usage+in+the+sequence+diagram
- 對應至C4 model的Code Diagram

元件設計範例

Example 1:

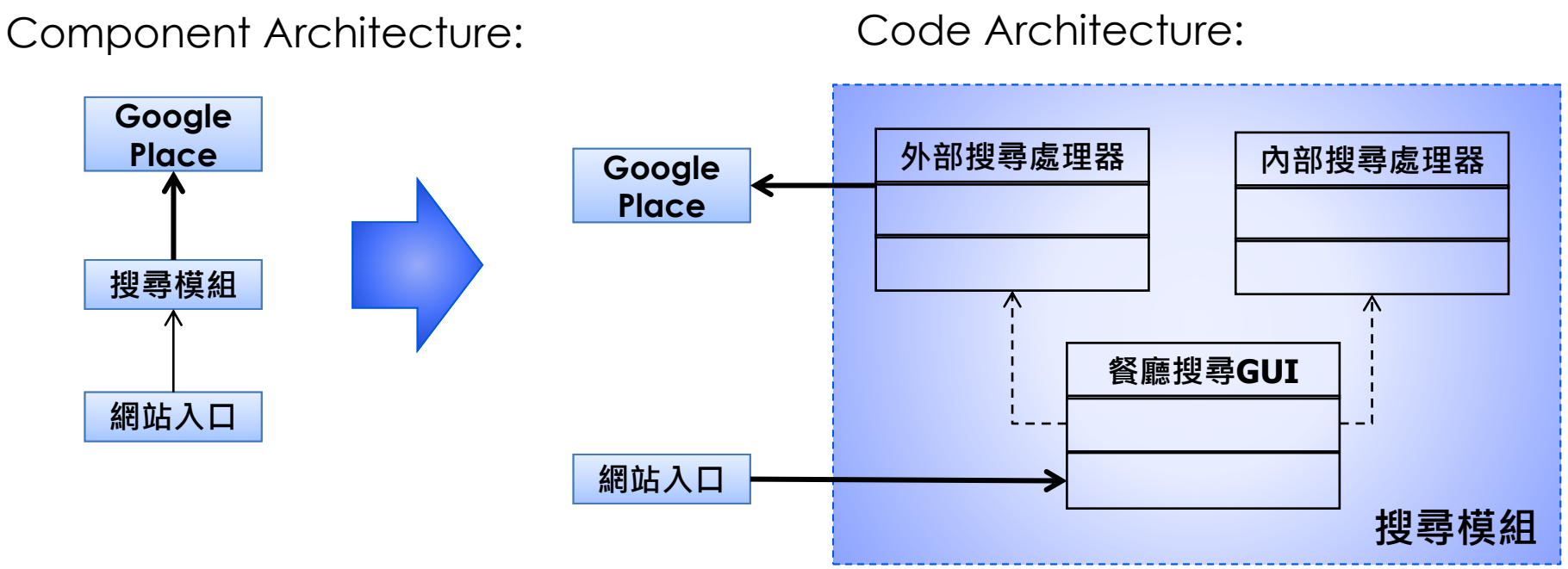


Example 2:



元件設計與架構圖1

- 類別圖設計需與上層架構圖一致：
  - 上層架構圖中的介面，到了類別圖應盡量維持



元件設計與架構圖2

- 上層架構圖中每個模組應該要往下拆解出一到多個單元(程式)，這些單元有可能是類別、網頁、JavaScript、後端程式(py或php)等。
- 請至少訂出要寫多少支程式(不須很細的method或field)，以及這些程式間的互動關係(dependency或association)。
- 請盡量表達從架構到類別圖的對應關係。可能會包含後面設計議題的一些結論。
- 若要表達特別元件分類，請用stereotype (如<<php>>)。

設計議題(Design Issue)

- Design Issue: 紀錄系統設計過程中遭遇的議題(Issue)
  - 議題內容
  - 可能解決方案(alternative solutions)
  - 最後解決方案(final solution)
  - 決策理由(rationale)

設計議題(Design Issue)：範例一

- 議題：Facebook之好友資訊應何時抓取，以能兼顧資料更新與執行時間？
- 可能解決方案：
  - 方案1：只在第一次登入時從Facebook取得好友名單，並紀錄於資料庫，之後只在使用者主動更新時，才更新好友名單。
  - 方案2：每次登入時都從Facebook更新好友名單，但更新時會做差異判斷，以新增/刪除好友
- 最終解決方案：
  - 方案2
- 決策理由：
  - 好友名單可能隨時都會變動，每次登入時需檢查使用者是否有新朋友，並將有在使用本軟體的好友頂置在名單上層中，方便使用者查看朋友的旅遊日誌。

設計議題(Design Issue)：範例二[1]

- 議題：需要決定如何儲存和管理使用者的購物車資料，以確保：
  - 未登入用戶也能使用購物車功能
  - 使用者登入後可以看到先前加入的商品
  - 購物車資料在跨裝置/瀏覽器時能同步
- 可能解決方案：
  - 方案1： 純瀏覽器端儲存
    - 使用 localStorage/sessionStorage 儲存購物車資料
  - 方案2：純後端資料庫儲存
    - 所有購物車資料都存在資料庫、未登入用戶用臨時 ID 識別
  - 方案3: 混合式儲存(最終方案)
    - 瀏覽器端暫存資料定期與後端同步

設計議題(Design Issue)：範例二2

- 最後解決方案
  - 方案3混合式儲存
  - 具體實作方式：
    - 前端使用 localStorage 儲存購物車資料
    - 每30秒與後端同步一次
    - 後端使用 Redis 儲存購物車資料(TTL 30天)
    - 未登入用戶使用 UUID 作為識別
- 決策理由：
  - 可滿足所有設計考量
  - 使用localStorage可提供即時反應
  - Redis快取可減輕資料庫負擔
  - 使用者操作無延遲感

設計議題(Design Issue)：其他素材

- 其他設計議題範例/素材：
  - System Design Interview: https://www.educative.io/courses/grokking-the-system-design-interview (有部分免費素材)
  - System Design 101： https://github.com/ByteByteGoHq/system-design-101

Any Question?

